Lux: Current Architecture Specification

Visual Output Surface for AI Agents — a System in Transition

July 2026 — v0.21.0

Contents

1 Overview **2**
- 1.1 Current and Target . . . 2
- 1.2 Companion Documents . . . 2
- 1.3 Design Principles . . . 3

2 System Architecture **3**
- 2.1 Component Map . . . 3
- 2.2 The Operations Facade . . . 4
  - 2.2.1 Typed Requests and the Never-Raising Parse . . . 5
  - 2.2.2 The Error Contract . . . 5
- 2.3 Data Flow . . . 5
  - 2.3.1 Write Path (Client → Hub → Display) . . . 5
  - 2.3.2 Event Path (User → Hub, two-tier D21) . . . 6
  - 2.3.3 Update Path (Incremental Patching) . . . 6

3 The Hub/Display Slice **6**
- 3.1 HubDisplay: the Authoritative Store . . . 6
- 3.2 The Element ABC . . . 7
- 3.3 Two-Tier Handler Dispatch (D21) . . . 7
- 3.4 Agent Subscribe / Publish . . . 7
- 3.5 Element-ABC and the Display Role . . . 8

4 Wire Protocol **8**
- 4.1 Framing . . . 8
- 4.2 Message Types . . . 8
  - 4.2.1 Client → Display . . . 9
  - 4.2.2 Display → Client . . . 9
- 4.3 Element Kinds . . . 9
- 4.4 Draw Commands . . . 10

5 Scene and Frame Lifecycle **11**
- 5.1 Frames Die With Their Scenes . . . 11
- 5.2 Scenes Survive Their Session . . . 11
- 5.3 Frame Time-to-Live . . . 11
- 5.4 The Frame Window . . . 12

6 Identity, Addressing, and Multi-Hub Topology **12**
- 6.1 Problem . . . 12
- 6.2 Identity Is a Path . . . 13
- 6.3 The Three-Rung Ladder . . . 13
- 6.4 Value Types . . . 13
- 6.5 Hub Identity on the Wire . . . 15
- 6.6 Every Aggregated Surface, Uniformly . . . 16
- 6.7 Aggregated Storage: `HubScopedStore` . . . 17
- 6.8 Multi-Hub Topology . . . 18
  - 6.8.1 What Already Works, Unmodified . . . 19

6.8.2 What Must Change . . . 19
6.8.3 Discovery, Same-Host . . . 20
6.9 Governing Invariant . . . 20
6.10 Reconciliation with Existing Design . . . 20

7 Cross-Host Transport and Trust **20**
7.1 Threat Model . . . 21
7.2 Resolving the Trust Fork: `HubId.hostname` Is Transport-Verified . . . 22
7.3 Client Host: Observed on the Transport . . . 22
7.4 Transport Specification . . . 23
7.5 Trust Anchor Providers: Pluggable, Not Fixed . . . 24
7.6 Coexistence with the Local Fast Path . . . 25
7.7 Discovery and Connection Lifecycle . . . 26
7.8 Authentication and Enrollment . . . 27
7.8.1 Provider 1 (Default): Self-Managed Personal CA . . . 27
7.8.2 Provider 2 (Optional): AWS Private CA (Managed) . . . 28
7.9 Rejected Alternatives . . . 29
7.10 Invariants . . . 29
7.11 z-spec Assessment . . . 30
7.12 Reconciliation with Existing Design . . . 30

8 Transport and Process **31**
8.1 The MCP Leg: Streamable HTTP . . . 31
8.2 The Loopback Policy . . . 31
8.3 The Display Socket Is Hub-Internal . . . 31
8.4 Display Socket Discovery and Auto-Spawn . . . 31
8.5 The Display Render Loop . . . 32

9 Client Surfaces **32**
9.1 MCP Tools . . . 32
9.2 REST Routes . . . 33
9.3 The Command-Line Tool . . . 34

10 Performance **34**
10.1 Frame Budget . . . 34
10.2 Critical Path . . . 35
10.3 Memory . . . 35
10.4 Known Limitations . . . 35

11 Security **35**
11.1 Trust Boundaries . . . 35
11.2 Render Function Consent . . . 35
11.3 Protocol Robustness . . . 35

12 Platform Integration **36**
12.1 macOS . . . 36
12.2 Linux . . . 36

13 Formal Models **36**

14 Test Architecture **37**

1 Overview

Lux is a visual output surface for Claude Code agents and local applications. An agent or an app describes a user interface as a tree of typed JSON elements and submits it to Lux; Lux installs that interface in one authoritative store and renders it in a native ImGui window.

The system is built from one engine fronted by thin client surfaces. The engine is split into two cooperating processes:

- The **Hub** (`luxd`) owns the authoritative interface state, runs the real interaction handlers, and coordinates the display. It is the single front door: every client talks to the Hub.
- The **Display** (`lux-display`) holds a full replica of the interface and renders it every frame. It captures user input and forwards each interaction back to the Hub. It never talks to a client directly.

Four client surfaces reach the Hub, and each is thin:

- The **library** surface is the operations layer itself — a direct in-process call into the engine.
- The **MCP** surface is a set of tools an agent calls; it speaks streamable HTTP to the Hub.
- The **REST** surface is a typed FastAPI application on the Hub; it is the front door for the command-line tool and any non-MCP caller.
- The **CLI** (`lux`) is an HTTP client of the REST surface.

1.1 Current and Target

This document describes the architecture *as the code stands today*. The canonical design intent lives in `docs/architecture/target/target.md`; where this document and that target disagree, the target defines the *direction* and this document reports the *present*. Each place the present diverges from the target is called out explicitly.

The code has converged substantially on the target. The Hub is authoritative, the operations layer is the single home of every capability, the MCP tools and REST routes are adapters over it, and the command-line tool reaches the Hub over REST. Two areas remain in transition:

- **Element migration.** Fifteen of the twenty-five element kinds are on the Element-ABC path (data and behavior on the class); ten kinds are still frozen dataclasses decoded by per-kind codecs. See Section §3.
- **The display's own state.** The Display still owns its render loop, its themes and window settings, and its diagnostic ring buffers. The Hub reaches those facts by proxying a read or write over its one connection to the Display (Section §9).
- **Multi-Hub addressing and cross-host transport.** Sections §6 and §7 specify a design — ACCEPTED, ratified by the operator 2026-09-05 (DES-089, DES-090) — for a Display aggregating more than one Hub, including Hubs on other machines. None of it is implemented yet: the code today assumes exactly one Hub connection, though nothing in the transport enforces that. The full bead map and dependency order are in the companion increment-of-work document, `docs/architecture/multi-hub-addressing-work.md`.

1.2 Companion Documents

The sibling files provide narrower supporting views:

- `target/target.md` — the canonical rewrite target. Start there for design intent; return here for current behavior.
- `one-code-path.md` — the design of the operations layer, the REST front door, the thin adapters, and the streamable-HTTP transport. It is the design record for most of what this document now describes as shipped (ADR DES-055).

- `target/introspection-api.md` — the verification and control surface an agent uses to inspect live Hub and Display state.

Four formal models hold parts of the system to a checked specification; they are cross-referenced where each applies and listed together in Section §13.

1.3 Design Principles

P1. **One engine, thin clients.** Every capability is one typed operation on the Hub. The library call, the MCP tool, the REST route, and the CLI command all run that same operation; none reimplements it.

P2. **The Hub is the authority.** Interface state that a client submits is owned by the Hub. The Display is a replica, and it loses every disagreement.

P3. **Protocol is the API surface.** An agent describes an interface as JSON; Lux renders it. There is no shared memory and no required Python import.

P4. **UI state crosses the wire; render calls do not.** Serialized elements, scene updates, and interaction messages travel between Hub and Display. ImGui calls stay inside the Display.

P5. **Whole-UI resend on change.** When Hub-side state changes, the Hub re-sends the whole affected interface and the Display replaces its copy. There is no incremental diff protocol.

2 System Architecture

2.1 Component Map

The package divides into the engine and its client surfaces. The engine is the operations layer, the Hub domain, the scene state, and the protocol; the surfaces are the MCP tools, the REST routers, the CLI, and the transport plumbing that carries each. The Display is the engine's second process.

Concern	File or package	Responsibility
Operations	`src/punt_lux/operations/`	The engine's front of house. One `Operations` facade composes concern classes — `SceneOperations`, `QueryOperations`, `MenuOperations`, `DisplayControlOperations`, `PubSubOperations`, `ConvenienceOperations`, `DisplayModeOperations`. Each method takes a typed request and returns a typed result or an `OpError`. Models live in `src/punt_lux/operations/models/`.
Hub domain	`src/punt_lux/domain/hub/`	The authoritative engine core. `HubDisplay` is the store; `HubReplicator` is the sole writer to the Display; `FrameLifecycle`, `FrameExpiry`, and `ExpirySweep` own the frame time-to-live; `HubReads` answers introspection; `HubMenuRegistry` owns the menu bar; `HubSceneWriter` applies updates and clears; `ClientRegistry` owns luxd's one display connection and the D21 dispatch.
Domain (pure)	`src/punt_lux/domain/`	The renderer-free and socket-free element model. `element_abc.py` is the `Element` ABC (data and behavior); `interaction.py` and `container_interaction.py` are the typed events; `handlers/remote_dispatch.py` wraps handlers for the two-tier path.

Concern	File or package	Responsibility
Replica	src/punt_lux/display/replica/	The Display's scene graph. SceneReplica owns the per-scene widget state; FrameBook owns the frames and the scene-to-frame and scene-to-owner maps; frame.py is the Frame window; MenuReplica owns the replicated menu state.
Protocol	src/punt_lux/protocol/	The wire contract: length-prefixed JSON framing, the FrameReader streaming buffer, the twenty-five element kinds in src/punt_lux/protocol/elements/, and the message families in src/punt_lux/protocol/messages/.
Display	src/punt_lux/display/	The ImGui render loop and coordinator. server.py, element_renderer.py, table_renderer.py, menu_manager.py, idle_screen.py, and texture_cache.py.
MCP tools	src/punt_lux/tools/	Thin adapters over the operations facade. tools.py and composite_tools.py are display-facing tools; subscribe_tools.py are the session pub-sub tools; server.py owns the FastMCP instance.
REST	src/punt_lux/rest/	The typed FastAPI surface. RestSurface composes concern routers (scenes, menus, display, config); status.py is the one OpError-to-HTTP table.
MCP transport	src/punt_lux/mcp_transport.py	luxd's MCP leg: streamable HTTP mounted at /mcp beside the REST routes. mcp_endpoint.py resolves session identity; mcp_session.py runs the per-session cleanup cascade; session_cleanup.py tears down a departed session's Hub state.
CLI transport	src/punt_lux/rest_client.py	LuxRestClient — the CLI's typed HTTP client of luxd's REST routes. rest_transport.py is the HTTP transport; rest_loopback.py is the in-process transport tests use.
Gateway	src/punt_lux/luxd.py	Builds the one FastAPI app on uvicorn, mounts the MCP leg and the REST surface, and starts the frame-expiry sweep on the event loop.
Display client	src/punt_lux/domain/hub/display_link.py	DisplayLink — the Unix-socket client of the Display. Constructed in exactly one place, src/punt_lux/domain/hub/clients.py, and an AST guard enforces that (Section §8).
Paths	src/punt_lux/paths.py	DisplayPaths — display socket discovery, PID lifecycle, and auto-spawn. src/punt_lux/hub_paths.py is the symmetric HubPaths for luxd's port and PID files.
Apps/Beads	src/punt_lux/apps/beads.py	BeadsBrowser — builds the beads issue table as an element tree. Pure: it holds no socket.
Config	src/punt_lux/config.py	ConfigManager — reads and writes the per-repo display-mode toggle in <repo>/.punt-labs/lux.md.
CLI	src/punt_lux/__main__.py	The Typer app: show, service, status, and installation commands.

2.2 The Operations Facade

Every capability the system offers is one method on the Operations facade (src/punt_lux/operations/facade.py). The facade composes seven concern classes, each owning one group of capabilities, and delegates each method to the right one. A caller — an MCP adapter, a REST route, or a test — holds one object and calls one method.

The facade binds no process singletons at import time. Its for_store classmethod receives every collaborator (the HubDisplay store, the replicator, the Hub, the client registry, the menu registry, the

display connection) from the composition root, so the operations are testable without the running process.

2.2.1 Typed Requests and the Never-Raising Parse

Each operation takes a typed request model and returns either the operation's own success model or a shared error. Validation is the operation's job, not the adapter's. A request model carries a never-raising `parse` classmethod: it validates raw arguments and returns either the request or an `OpError`, so an adapter that builds a plain mapping and calls `parse` can never raise past its string contract. The operation accepts the parse result and passes an `OpError` straight through.

The consequence is one schema per capability, applied at one point. A bad `layout` becomes an `invalid_request` error inside `parse`; the MCP adapter renders it to the legacy status string, and a REST route renders it to an HTTP 422. The rules live in one place; only the rendering differs by surface.

2.2.2 The Error Contract

Every operation may return `OpError` (`src/punt_lux/operations/models/common.py`), a frozen model tagged `kind="error"` so a result cannot be both a success and a failure. The `code` is a closed set the caller branches on; the `reason` is the human sentence for logs and messages.

Code	Meaning
`display_unavailable`	The display process is not running.
`timeout`	A proxied round-trip to the display exceeded its bound.
`rejected`	The engine refused the caller's request (the caller's fault).
`invalid_request`	The request itself did not type-check.
`not_found`	The named scene or resource does not exist.
`fault`	An engine-side failure: a malformed display reply, an unreadable config file.

The REST surface maps each code to one HTTP status through a single shared table (`src/punt_lux/rest/status.py`): `invalid_request` to 422, `not_found` to 404, `rejected` to 409, `fault` to 502, `display_unavailable` to 503, and `timeout` to 504. The mapping is a table, not per-route logic, so a new operation inherits it for free. `LuxRestClient` reads the same table in reverse, so the CLI observes the same contract from the other end.

2.3 Data Flow

Three data paths connect a client to the Display.

2.3.1 Write Path (Client → Hub → Display)

1. A client calls a capability — the `show` MCP tool, a `PUT /scenes/{id}` REST call, or `lux show beads`.
2. The surface builds a plain mapping and calls the request builder, then calls one operation. For `show` that is `Operations.render`.
3. The operation validates the request, then installs the scene into `HubDisplay` — the authoritative store — under the store lock, indexing every element and its owner.
4. The operation marks the scene dirty on the `HubReplicator` and returns its typed result at once. The client's call does not wait on the Display.
5. The `HubReplicator`, the one background worker that writes to the Display, drains the dirty set, serializes the scene, and sends it over luxd's single socket connection.
6. The Display receives the scene, replaces its replica, and renders it every frame.

2.3.2 Event Path (User → Hub, two-tier D21)

The event path is where the Hub/Display split is clearest. The Display does not run interaction handlers; it forwards each interaction to the Hub, where the real handler runs.

1. When the Display installs a scene, it wraps every handler on each element, replacing each event bucket with a `RemoteDispatchGroup` that captures the socket-send closure.
2. The user interacts with a widget. The renderer fires the typed event on the Display's element copy, and the wrapped handler runs instead of the real handler.
3. The wrapper builds one `RemoteEventHandlerInvocation` (`element_id`, `action`, `event_kind`, `value`, `ts`, `scene_id`) and sends it to the Hub. One interaction produces one message per element/event bucket, however many handlers that bucket holds.
4. The Hub receives the message in `ClientRegistry._hub_interaction_dispatch`, resolves the authoritative element by `(scene_id, element_id)`, checks its owner, builds the typed event, and fires the real handler chain once on its own copy.
5. If the handler mutated Hub state (for example a dialog dismissed itself), the Hub marks the scene dirty; the replicator re-sends the whole scene and the Display renders the current tree.

A `frame_close` action is not an element interaction: the Display sends it when the user closes a frame, and the Hub answers by removing that frame's scenes so both tiers agree the frame is gone (Section §5).

2.3.3 Update Path (Incremental Patching)

1. A client calls `update(scene_id, patches)`, reaching `Operations.update`.
2. The operation applies the patch batch to `HubDisplay` through `HubSceneWriter`: a `set` patch changes fields on the target element, a `remove` patch drops it.
3. The operation marks the scene dirty; the replicator re-sends the whole scene, and the Display renders the result.

3 The Hub/Display Slice

This section describes the Hub-authoritative core and the Element-ABC path the migration is converging on. It is where new element work lands and the reference for how the whole system is meant to behave once migration completes.

The domain code in `src/punt_lux/domain/` imports no ImGui, no socket, and no JSON. Adapters wrap it: the Display renders it, and the replicator moves its serialized elements over the wire.

3.1 HubDisplay: the Authoritative Store

`HubDisplay` (`src/punt_lux/domain/hub/hub\_display.py`) is the Hub's single authoritative surface for scene state. It is a facade over typed collaborators, each with one responsibility:

Collaborator	Responsibility
`ElementIndex`	`(scene_id, element_id)` → `Element` lookup.
`OwnerTracker`	Every element's owning connection; the interaction path checks it, and the disconnect path walks it.
`RootRegistry`	Scene-root elements that take part in the property-observer cascade.
`ChildIndex`	Parent-to-children edges captured at install time so cascade removal drops every descendant in one walk.
`HubClientRegistry`	The connections currently registered as Hub clients.
`FrameLifecycle`	Each scene's frame presentation, its optional TTL deadline, and its teardown — all under the store lock (Section §5).
`HubReads`	The authoritative introspection reads that answer `inspect_scene` and `list_scenes`.

A write dispatches on the typed `Update` sum — `AddElement`, `SetProperty`, `RemoveElement` — and enforces ownership: a connection cannot mutate or evict another connection's elements. `AddElement` recurses into composite children so a button buried inside a dialog is indexed and later click-resolvable. Re-showing a scene removes the connection's previous roots and installs the new ones through the same write path, so ownership, root observers, and child indexes rebuild in one place.

3.2 The Element ABC

The migrated kinds subclass the `Element` ABC in `src/punt_lux/domain/element\_abc.py`. The ABC carries data *and* behavior:

- **Composite render template.** `render()` is a template method and is never overridden; a composite overrides `_children()` to return its children, and a leaf inherits the empty default.
- **Handler registry.** `add_handler`, `remove_handler`, and `fire` key handlers by `Event` subclass. `fire` snapshots the bucket so a handler that mutates the registry mid-dispatch cannot affect the in-flight call.
- **Property-observer surface.** `add_observer`, `mark_removed`, and `removed` let a parent composite react to a child being removed. All three removal paths — an agent `RemoveElement`, a component self-dismiss, and a connection disconnect — route through the one `mark_removed` method.
- **Native serialization.** `__reduce__` and `__setstate__` let an element cross the Hub-to-Display boundary. Hub-only bookkeeping (the observer closures that reference `HubDisplay`) is dropped on serialization; the Display is a replica and does not need Hub observers. Handlers survive so the Display can wrap them for remote dispatch.

3.3 Two-Tier Handler Dispatch (D21)

The Hub says: "I am sending you a copy of an interface. If anyone interacts with it, I will do the work." The Display says: "I will render that copy and forward interactions back to the Hub."

Both tiers hold the same element state and both call `elem.fire(event)`. The difference is what the handler does:

- **Hub:** the real handler runs — `call_model`, `publish`, `mark_removed`.
- **Display:** the wrapped handler sends one `RemoteEventHandlerInvocation` to the Hub, which resolves the authoritative element, checks its owner, and fires the real event once.

Four event kinds cross this boundary today:

Event	Wire kind	Fired by
`ButtonClicked`	`button_clicked`	a button press.
`ValueChanged`	`value_changed`	a checkbox, slider, combo, radio, selectable, or text-input change.
`HeaderToggled`	`header_toggled`	a `collapsing_header` opening or closing.
`TabChanged`	`tab_changed`	a `tab_bar` switching tab.

`ButtonClicked` and `ValueChanged` are in `src/punt_lux/domain/interaction.py`; `HeaderToggled` and `TabChanged` are in `src/punt_lux/domain/container\_interaction.py`. The Hub builds the typed event from the wire kind and value, denying by default when the value has the wrong shape. Grouping is per element/event bucket: a button with three handlers still produces one remote message, and the Hub replays the full chain once.

3.4 Agent Subscribe / Publish

The Hub also owns an application-level publish/subscribe channel (`src/punt_lux/domain/hub/hub.py`), distinct from the element-observer mechanism above; the two share no machinery.

Every operation is scoped to a connection. A connection cannot see, touch, or publish into another connection's topics. `subscribe` registers the caller's outbound writer against a topic; `publish` fans an

`ObserverMessage` payload out to that connection's subscribers using snapshot-then-iterate, so a slow handler cannot stall concurrent publishes. Topics — `openTicket`, `closeTicket`, `markTicketInProgress` — are defined by the app or agent, not by Lux. This is the business-event channel, not the interface-replication path.

3.5 Element-ABC and the Display Role

Role	What it covers
Element-ABC	All twenty-five element kinds. Every kind decodes, mutates, and renders through the Element-ABC path; the pre-migration frozen-dataclass model and its fork were deleted in B7. Three kinds — `group`, `collapsing_header`, `tab_bar` — are ABC containers holding ABC children. `HubDisplay` authority with ownership and cascade removal, the D21 four-event path, the frame lifecycle, and Agent Subscribe/Publish are all in this column.
Display	The Display process runs the D21 event path and owns rendering, scene tabs, frames, and its own diagnostic buffers — a decomposition of the one engine, not a second authority.

The direction is fixed by `target/target.md`; the pace is set by migrating one element family at a time, container plus primitives first (the migration plan, DES-041).

4 Wire Protocol

4.1 Framing

Every message is a length-prefixed JSON frame:

```
+-----------------+---------------------------+
| 4 bytes (uint32) | N bytes (UTF-8 JSON) |
| big-endian | payload |
+-----------------+---------------------------+
```

Maximum payload size is 16 MiB (2^{24} bytes). The `FrameReader` (in `src/punt_lux/protocol/\_\_init\_\_.py`) accumulates partial reads into a byte buffer and yields complete frames via `drain()` (raw dicts) or `drain_typed()` (Message dataclasses). An oversize frame raises `ValueError`, which the socket layer converts to a client disconnect.

This framing is the Hub-to-Display wire. A client reaches the Hub over HTTP (MCP streamable HTTP, or REST), not over this socket; the socket is Hub-internal (Section §8).

4.2 Message Types

The message layer is split across six families in `src/punt_lux/protocol/messages/`: `scene`, `lifecycle`, `menu`, `introspect`, `observer`, and `remote_invocation`. There is no standalone `interaction` family — a user interaction travels as `RemoteEventHandlerInvocation` from the `remote_invocation` module. A `MessageRegistry` (`src/punt_lux/protocol/messages/registry.py`) is the wire dispatch table, populated at import time by each family module; a test can build an isolated registry without touching the module default.

4.2.1 Client → Display

Type	Purpose
`SceneMessage`	Replace or add a scene. Fields: `id`, `elements[]`, `title`, `layout`, `frame_id`, `frame_title`, `frame_size`, `frame_flags`, `frame_layout`.
`UpdateMessage`	Incremental patches. Fields: `scene_id`, `patches[]` (each: `id`, `set`, `remove`).
`ClearMessage`	Remove scenes and reset state.
`PingMessage`	Heartbeat. Field: `ts`.
`ConnectMessage`	Client identity handshake. **Current fields:** `name`, `kind` (`"hub"` or `"test"`). **Designed, not yet shipped:** a dedicated, required `hub_id` field, populated by every connecting kind including `"test"` — Section §6.
`MenuMessage`	Set the menu bar.
`RegisterMenuMessage`	Register per-connection menu items.
`ThemeMessage`	Apply a theme by name.
`QueryRequest`	A control or introspection request the display answers. Fields: `method`, `params`.

4.2.2 Display → Client

Type	Purpose
`ReadyMessage`	Handshake on connect: protocol version and capabilities.
`AckMessage`	Acknowledges a scene or update: `scene_id`, `ts`, optional `error`.
`RemoteEventHandlerInvocation`	A user interaction forwarded to the owning Hub: `element_id`, `action`, `event_kind` (`button_clicked`, `value_changed`, `header_toggled`, `tab_changed`), `value`, `ts`, optional `scene_id`.
`ObserverMessage`	A connection-scoped business event for the Agent Subscribe/Publish path: `topic` + `payload`.
`PongMessage`	Ping response.
`QueryResponse`	A control or introspection response: `method`, `result`, optional `error`.

An unknown message type deserializes as `UnknownMessage` for forward compatibility rather than disconnecting the peer.

4.3 Element Kinds

The wire layer exposes twenty-five element kinds plus the non-element `Patch` payload used by `UpdateMessage`. All twenty-five are on the Element-ABC path (Section §3); the pre-migration frozen-dataclass model and its `ElementCodec` dispatch table were deleted in B7. The `Element` union is assembled in `src/punt_lux/protocol/elements/\_\_init\_\_.py`.

Every kind serializes two ways. Its Level-1 form is a plain JSON dict through its per-kind codec; its wire form crossing the Hub-to-Display boundary is a base64-encoded pickled entry inside the `SceneMessage` codec, which preserves the Hub-side handlers the Display re-wraps.

Category	Element	Notes
`basics`	`text`	Content plus style (heading, caption, success, error, code); optional `color` hex string. ABC.
	`markdown`	CommonMark via imgui_md. ABC.
	`image`	File path or base64 data. ABC.
	`separator`	Visual divider. ABC.
	`progress`	Fraction bar with label. ABC.

Category	Element	Notes
	spinner	Loading indicator. ABC.
inputs	button	Label, action, disabled; arrow and small variants. ABC.
	slider	Min, max, value, integer mode. ABC.
	checkbox	Boolean toggle; fires ValueChanged. ABC.
	combo	Dropdown selection. ABC.
	input_text	Single-line text input. ABC.
	radio	Radio button group. ABC.
	input_number	Numeric input with step buttons and clamping. ABC.
	color_picker	RGB/RGBA hex picker. ABC.
	selectable	Click-to-select item; fires ValueChanged. ABC.
dialog	dialog	Dialog root with title, children, handlers, and a model-backed lifecycle. ABC.
layout	group	Row or column layout. ABC container.
	collapsing_header	Expandable section; fires HeaderToggled. ABC container.
	tab_bar	Tabbed container; fires TabChanged. ABC container.
	window	Movable, resizable floating panel. ABC container.
	tree	Recursive node hierarchy; flat flag. ABC.
	modal	Modal popup; emits "closed". ABC container.
graphics	draw	Canvas with typed draw commands (§4.4). ABC.
	plot	ImPlot: line, scatter, bar series. ABC.
table	table	Columns, rows, built-in filters and detail panel. ABC.

4.4 Draw Commands

The draw element holds a tuple of typed DrawCommand records. Each command is a frozen, slotted dataclass that satisfies the DrawCommand Protocol (TYPE: ClassVar[str], to_wire(self) -> dict, from_wire(cls, d, *, ctx) -> Self).

Module	Record	Geometry
draw_commands_line	Line	Start, end, color, thickness.
	Polyline	Point list, color, thickness, closed flag.
draw_commands_shape	Rect	Top-left, bottom-right, color, thickness, rounding, filled.
	Circle	Center, radius, color, thickness, filled.
	Triangle	Three points, color, thickness, filled.
draw_commands_curve	BezierCubic	Four control points, color, thickness.
draw_commands_text	TextGlyph	Position, content, color.

Shared value classes live in draw_values.py and draw_bounds.py: Point2 (a validated $[x, y]$ pair), Color (#RRGGBB / #RRGGBBAA hex), Thickness (a strictly positive stroke width), Radius (strictly positive), and Rounding (non-negative). The WireContext in draw_wire.py carries the per-error prefix and

the require-or-raise helpers. `DrawCommandDecoder` in `draw_decoder.py` dispatches on the wire `cmd` string to the right `from_wire` classmethod; `DrawElement.commands` is typed `tuple[DrawCommand, ...]` and the renderer dispatches via a typed `match` statement.

5 Scene and Frame Lifecycle

A scene targets a named *frame* — a persistent ImGui window that organizes the workspace. The lifecycle answers three questions the same way on both tiers: when a frame appears, when it disappears, and how long it lives without a refresh.

5.1 Frames Die With Their Scenes

A frame exists to show scenes. When a frame's last scene is removed, the frame closes; a frame still holding other scenes stays open.

The Hub blanks a scene by pushing it with no elements. The Display treats an empty-element push as a removal, not as an empty window: it dismisses the scene from its frame and closes the frame once it holds nothing — every scene is framed, so there is no separate unframed removal path. This is the amended contract the legacy display-server model now checks (Section §13).

When the user closes a frame on the Display, the Display sends a `frame_close` action. The Hub removes every scene that frame held from `HubDisplay` and marks each dirty, so the authoritative store agrees with the window the user already closed. The blank the replicator then sends is idempotent with that close.

5.2 Scenes Survive Their Session

A scene outlives the connection that created it. When an MCP session ends — the client disconnects, or the SDK reaps it on idle — the Hub forgets that connection as a client and releases its subscriptions and inbox, but it leaves the scenes installed. The scenes stay owned by the departed connection id, so a later explicit removal — a frame close, a `clear`, or a TTL expiry — can still reference the owner. Nothing is blanked, so no repaint is marked. A session's death does not erase what it drew.

5.3 Frame Time-to-Live

A frame may carry a time-to-live. Shown with a `ttl_seconds`, it is removed once that many seconds pass with no re-show refreshing it. Three small components implement this, all in `src/punt_lux/domain/hub/`:

- `FrameExpiry` owns the bookkeeping: a `frame_id` to a deadline on a monotonic clock (a duration, immune to wall-clock adjustment). It answers which frames are due now and how long until the soonest one. It holds no lock.
- `FrameLifecycle` owns each scene's frame presentation, its deadline, and its teardown, all under the one store lock. Recording a presentation and arming its deadline is a single locked step, so a re-show that refreshes a frame cannot install it yet forget to re-arm it.
- `ExpirySweep` is the task on luxd's event loop that enforces the deadlines. It sleeps until the soonest deadline (capped at a one-second idle poll), sweeps the frames now due, and marks their scenes dirty so the replicator blanks both tiers — the same path a manual frame close takes. It is one more store mutator on the loop, not a second timer thread.

Because arming a deadline and expiring a frame both run under the store lock, a frame re-armed with a fresh TTL is never torn down by a stale deadline: the re-show and the sweep serialize, and whichever runs second sees the other's effect. A re-show carrying no TTL makes the frame permanent. This atomicity is the safety property the frame-expiry formal model checks (Section §13).

A due frame is peeked, not consumed: the sweep tears each frame down and only then disarms its deadline, so a tear-down that raises leaves the frame armed to retry on the next sweep rather than stranded — torn down on the Hub but never blanked on the Display. Frames are isolated from one another: a frame whose tear-down raises is logged and skipped, and every frame that did tear down is still repainted.

5.4 The Frame Window

The `Frame` class (`src/punt_lux/display/replica/frame.py`) is the Display's window for a set of scenes. `FrameBook` (`src/punt_lux/display/replica/frame\_book.py`) owns the frame collection and the scene-to-frame and scene-to-owner maps, split out of `SceneReplica` so each has one job.

Field	Type and Purpose
`frame_id`	`str` — unique identifier.
`title`	`str` — window title bar.
`scenes`	`dict[str, SceneMessage]` — the frame's scenes, keyed by `scene_id`.
`scene_order`	`list[str]` — tab ordering.
`active_tab`	`str | None` — the visible scene.
`minimized`	`bool` — docked to the bottom bar.
`cascade_index`	`int` — stagger offset for initial placement.
`initial_size`	`tuple[int,int] | None` — first-use size hint.
`flags`	ImGui window flags (`no_resize`, `no_collapse`, `no_title_bar`, `no_background`, `no_scrollbar`, `auto_resize`).
`layout`	`Literal["tab", "stack"]` — multi-scene composition mode.

Left-clicking the docking background opens the World menu, which is the menu bar brought to where the user clicked. Both surfaces render one menu model (`src/punt_lux/display/menus/model.py`) that `MenuReplica` composes each frame: the display's own Lux, Windows, and Help menus, then the bars agents submit through `set_menu`, then one submenu per live session that registered a callback. Neither surface builds entries of its own (`MenuBar` and `WorldPanel` in `src/punt_lux/display/menus/projections.py` each take the model they render), so the two cannot show different menus, and a click on a leaf sends the same `RemoteEventHandlerInvocation` whichever surface it came from. A minimized frame appears as a button in a bottom dock bar; clicking it restores the frame.

6 Identity, Addressing, and Multi-Hub Topology

Design status: DES-089 in `DESIGN.md`, Status: ACCEPTED — ratified by the operator 2026-09-05. The addressing model below is evaluator-accepted design, not yet implemented; the code today runs a single Hub connection to the Display in practice, though nothing in the transport enforces that. The implementation plan and bead map live in the companion increment-of-work document, `docs/architecture/multi-hub-addressing-work.md`.

6.1 Problem

The Display is an aggregator: it renders content it did not produce, from producers it does not control, from a Hub that may itself aggregate many agents and many users — and, per the operator's ruling, may itself be more than one Hub. Two failures follow when a lower layer's identity is used, unqualified, at a higher layer that can see more than one of that lower layer.

The first is visible: Dear ImGui raises "N visible items with conflicting ID" when two rendered widgets share a label, because ImGui uses the label as both display text and identity unless told otherwise. Four session panels all titled "Vox" collide the moment a second one exists.

The second is silent, and it is not hypothetical — it already happened once, at the layer below this one. `HubDisplay` used to store every scene and frame under the literal string a client submitted; two connections choosing the same `scene_id` did not error, the second silently evicted the first's root. The fix was `ConnectionScopedId`: compose the store key from the writing connection's own id, so collision becomes unrepresentable rather than merely checked.

The Display's own storage has the identical shape of bug, one layer up, and it is live in the tree today. `FrameBook` holds `_scene_to_frame` and `_scene_to_owner` as flat dictionaries keyed by the bare Rung-2 scene id string, spanning every connection the Display has ever seen. `MenuReplica` holds `_callback_menus`, replaced wholesale on every push. Neither carries a dimension for "which Hub sent this." `ConnectionId` is a deterministic hash of a client's self-declared fields, computed independently by

each Hub process with no cross-Hub coordination — two different Hubs can and eventually will mint the identical `ConnectionId` for two different real clients. The day a second Hub connects, its scene push can silently clobber the first Hub's scene under the identical key, by the same mechanism DES-086 already found and fixed once.

6.2 Identity Is a Path

Each layer that aggregates content knows only what it itself produced. Only the layer above a given aggregator can see that there might be more than one of it: an applet knows its own label and nothing else; a Hub knows its own live connections and nothing about other Hubs; the Display is the one layer that can see more than one Hub. The consequence is structural, not a style preference: **each layer must namespace what is below it, blind to what is above it.** Applied recursively down the stack, an item's true identity is not a label — it is the path from the outermost aggregator that can see it, down to the producer's own name for it. That is DES-089's governing phrase.

6.3 The Three-Rung Ladder

Three rungs exist. The first two are already shipped, in two different mechanisms that share one shape. The third does not exist in code today.

```
Rung 1 -- Producer label unscoped, freely reused
  "Vox", "Beads", "music-player"
        |
        | namespaced by the Hub that owns the connection (DES-058, DES-086)
        v
Rung 2 -- Hub-connection scope unique within ONE Hub's own registry
  ConnectionScopedId / CallbackInvocation: {connection_id}\x1f{local_id}
        |
        | namespaced by the Display, across every Hub it holds (this design)
        v
Rung 3 -- Hub-of-origin scope unique across every Hub the Display holds
  LuxAddress: {hub}\x1f{connection_id}\x1f{local_id}
```

Rung	Owned by	Scope of uniqueness	Shipped mechanism
1. Producer label	The applet/agent itself	None — every producer may reuse any string	`register_callback(id, label)`; a caller's `scene_id`/`frame_id`
2. Hub-connection scope	The Hub, over its own live connections	Unique within one Hub process's registry, not globally	`ConnectionScopedId` (scenes/frames, DES-086); `CallbackInvocation` (menu leaves, DES-058)
3. Hub-of-origin scope	The Display, over every Hub it holds	Unique across every Hub connection the Display currently holds	Does not exist yet — this design specifies it

Rung 2's own scope claim is the load-bearing fact Rung 3 rests on: `ConnectionId` is computed independently, per Hub process, from fields the client declares to that Hub. Nothing makes it globally unique — a single Hub has no way to know about, let alone coordinate with, any other Hub. That is exactly why Rung 3 must exist: the one layer that can see more than one Hub is the one layer obligated to disambiguate them.

6.4 Value Types

Identity is an object, not a concatenated string. Two small value classes compose into the one type that answers "what is this, uniquely and legibly": `Rung` (one level's key and label) and `LuxAddress` (the three rungs, composed).

```
@final
@dataclass(frozen=True, slots=True)
class Rung:
```

```
    """One level of a LuxAddress: a stable uniqueness key and a human
    label. ``key`` is opaque and never elided from the hidden id -- two
    Rungs with the same ``key`` are the same thing by construction.
    ``label`` is what a human reads, shown only when this rung's
    ambiguity requires it."""
    key: str
    label: str


@final
@dataclass(frozen=True, slots=True)
class LuxAddress:
    """The Display's complete identity for one item it aggregates.
    Composed of the three rungs, outermost first. Never round-tripped:
    nothing parses a LuxAddress back into its parts. It exists only on
    the Display side of the wire, purely to answer "is this the same
    item" and "how do I show it."""
    hub: Rung # Rung 3 -- which Hub connection produced this
    connection: Rung # Rung 2 -- which connection on that Hub produced this
    leaf: Rung # Rung 1 -- the producer's own name for the item

    @property
    def hidden_id(self) -> str:
        """The ImGui identity: every rung's key, joined, never elided."""
        return ID_SEPARATOR.join(
            (self.hub.key, self.connection.key, self.leaf.key)
        )

    def title(self, *, hub_ambiguous: bool, connection_shown: bool) -> str:
        """The visible title: the leaf always, higher rungs when the
        caller says. The Hub rung is shown only while ambiguous; the
        connection rung whenever the item has a label to show (DES-100).
        Both are caller answers, not carried on the address."""
        shown = list(
            compress(
                (self.hub.label, self.connection.label),
                (hub_ambiguous, connection_shown),
            )
        )
        shown.append(self.leaf.label)
        return " :: ".join(shown)
```

DES-100 amends the connection rung's elision. DES-089 elided every rung above the leaf while it was unambiguous. The operator ruled (2026-10-22) that the connection rung is instead shown on *every* frame with an owning session, one session or many: the same applet runs in many repositories by design, so a bare `Board` does not say whose board it is. There is no collision test at Rung 2. The Hub rung keeps DES-089's rule. The label itself is the Hub's roster name for the owning session (DES-064), sent with each scene as `SceneMessage.frame_scope_label`; a frame adopts a label only when one is sent, so a later push for a departed owner never blanks it, and only the empty-push eviction of the frame's oldest scene takes the Hub's fresh answer, absence included. When a departure renumbers the roster, the Hub re-pushes the scenes of every survivor whose label moved in the same store-locked step as the registry write, so titles follow by push.

`title()` joins on `" :: "` rather than `"/"`: a slash reads as a filesystem or URL segment — plausibly clickable, parseable, navigable — to the developer/agent audience this title is shown to, and no part of a `LuxAddress` title is any of those things. `"::"` is the namespace-qualification notation that audience already reads as "scope, not a route." `hidden_id` is `connection.key` joined with `leaf.key` on `ID_SEPARATOR` — exactly the string `ConnectionScopedId.compose` or `CallbackInvocation.menu_id` already produce — with the Hub dimension prepended on top, rather than re-derived.

`HubId` is the value type a Hub's own identity resolves to: what makes one Hub connection distinct from another, and, per the operator's cross-host ruling, distinct from a Hub on a different machine, not merely a different process on the same one.

```
@final
@dataclass(frozen=True, slots=True)
class HubId:
    """A Hub connection's own stable identity -- network host plus
    process. Two independent uniqueness axes, kept as two fields
    because they are disambiguated by two different mechanisms:
    ``hostname`` distinguishes one machine from another (FQDN via
    socket.getfqdn(), not socket.gethostname() -- a bare hostname is
    not guaranteed unique across a network the way it is on one box);
    ``pid`` distinguishes one process from another on the same
    machine, the same shape of fix AppletIdentity already uses for two
    applets sharing one session (DES-067's #{session_pid} token).
    ``pid`` is not network-meaningful and only ever breaks a tie
    within one already-identified host. Self-reporting its own FQDN is
    as far as this class goes, and it is trustworthy on the AF_UNIX
    leg only because the socket directory's 0700 permission already
    bounds who can open a connection to declare a HubId in the first
    place -- that argument does not carry across a network, which is
    exactly why cross-host needs mTLS (see Cross-Host Transport and
    Trust)."""
    hostname: str
    pid: int

    @classmethod
    def current(cls) -> Self:
        """This process's own HubId, as declared to the Display."""
        return cls(socket.getfqdn(), os.getpid())

    @property
    def wire_token(self) -> str:
        """The string this identity declares on the wire -- compared
        for equality only, never parsed."""
        return f"{self.hostname}{ID_SEPARATOR}{self.pid}"
```

Same-host duplicates keep the DES-064-style `(n)` numbering: two Hubs sharing one `hostname` (the same machine) disambiguate by `pid`, then render as `pembroke`, `pembroke (2)`. Only the *source* of the `hostname` field's trustworthiness changes, and only cross-host — see §7.

6.5 Hub Identity on the Wire

The addressing design's own first round left this as an open fork — reuse `ConnectMessage.name`, or add a dedicated field — and recommended reuse, following DES-067's precedent for the same shape of tradeoff. **The operator ruled the opposite way (2026-09-04): a dedicated `hub_id` field.** `name` keeps its existing, narrower meaning — "what a human calls this connection," the string a frame title or a menu already attributes content to — and does not also carry the Hub's own machine-and-process identity.

Second ruling (2026-09-05): `hub_id` is required, every connecting kind populates it, and the earlier `Optional` is withdrawn. The first round of this field made `hub_id` an `Optional[str]`: present for `kind="hub"`, absent for `kind="test"`, with a runtime hard-fail standing in at every Display-side store for the type-level guarantee the `Optional` declined to make. The operator's correction: a `kind="test"` connection is a stand-in for a Hub on this leg — it runs the identical accept path, the identical `ReadyMessage`/`ConnectMessage` exchange, and the identical eventual `AddressBook` bookkeeping a real Hub does — so it should present a `HubId` exactly as a real Hub does, not decline to. "This leg" is the same-host `AF_UNIX` leg specifically, the only leg `kind="test"` is ever sent on; §7 rejects `kind="test"` outright on the cross-host listener. A *stub* `HubId` (a synthetic localhost/test token, built the same way §6.4 already builds a real one) satisfies the identical contract. Once every connecting `kind` populates `hub_id`, there is no connection on the same-host `AF_UNIX` socket that legitimately lacks one, and both the `Optional` and the runtime discriminated check that used to stand behind it go away:

```
@dataclass(frozen=True, slots=True)
class ConnectMessage:
    """Client identifies itself to the display server. ``name`` is
    display attribution -- unchanged, still the string frame titles
```

```
and menu namespaces read. ``hub_id`` is a separate concern: this
connection's own HubId.wire_token. Every kind populates it,
including "test" -- a test connection is a same-host-only stand-in
for a Hub on the AF_UNIX leg and carries a stub HubId, never an
absence of one. The cross-host listener never accepts kind="test"."""
name: str
kind: Literal["hub", "test"]
hub_id: str
type: Literal["connect"] = "connect"
```

`kind` keeps exactly its existing, narrower job — `reject_scene_if_test_kind` still rejects test-origin content — and nothing else. Identity and that one behavioral gate are orthogonal: `hub_id` no longer varies by `kind` at all, so there is no discriminated shape left for `kind` to encode. **This retires, rather than defers, the earlier finding that the `Optional[str]` was a discriminated state standing in for a type the design had not yet given it** (Rule 5 of the OO standard, `punt-labs/.claude/rules`). The fix is not a `HubConnectMessage`/`TestConnectMessage` split held in reserve as a follow-on: a message-type split earns its keep only when the variants' *required fields* genuinely differ, and after this change they do not — every connecting kind carries the identical field set, and `kind` is a plain `Literal` tag selecting later behavior, exactly the ordinary case a `Literal` field exists for. Splitting the message here would buy two classes serializing an identical shape, for no discriminated behavior gained. The smell Rule 5 warns against is not the presence of a `kind` field; it is a field whose *meaning* silently changes what else is required or permitted. With `hub_id` required across the board, `kind` no longer has that shape, so a single `ConnectMessage` is the cleaner — and now the only motivated — expression.

Wire compatibility and multi-Hub correctness are not the same question, and making `hub_id` required at the type level moves the failure mode earlier rather than removing it. A sender's dict that omits `hub_id` now fails at *decode* — `ConnectMessage.from_dict` raises constructing the dataclass — rather than decoding successfully and being caught by a separate runtime check. This is the one honest residual of this design, and it is acceptable, in fact preferable: an un-adopted external sender (a pre-`W2` vox or z-spec process that has not yet added `hub_id` to the `ConnectMessage` it sends) fails loudly at connect time once this change lands, rather than decoding and then silently degrading. A Hub connection missing `hub_id` was already invalid under this design before this ruling — only the *point* at which it is caught moves, from a validation check after decode to the decode itself. Failing loudly with a message that amounts to "adopt `hub_id`" is a clearer signal than decoding successfully and failing a validation check three lines later, and there is no silent-degradation path being traded away to get it. `hub_id`'s adoption was never going to be a change one repo rolls out ahead of the others regardless of this ruling: vox and z-spec both run their own Hub connections (DES-063's applet model) and must add `hub_id` to the `ConnectMessage` they send in lockstep with this change landing here, per the org's cross-repo breaking-change protocol (notify, agree, land together, verify end-to-end, release together). See the increment-of-work document for the coordination sequence.

6.6 Every Aggregated Surface, Uniformly

The failure mode is not per-surface; the same mistake recurs at the next surface unless one mechanism owns it. Four surfaces the Display aggregates today or imminently:

Surface	Rung 1 (today)	Rung 2 (today)	Rung 3 (design)	Current leaf renderer
Menus (Clients, Windows)	`register_callback(label, ...)`	`CallbackInvocation.menu_id`	`hidden_id` on `MenuItem`	`MenuItem(label, activate)` — keys on the bare label today
Frames/windows	a caller-named `frame_title`/`frame_id`	`ConnectionScopedId`	`hidden_id` on the ImGui window id	`AddressRendering.frame_title` over `Frame.heading` (title plus scope label); the window id stays `hidden_id`
Scenes	a caller-named `scene_id`	`ConnectionScopedId`	`HubScopedKey` in a `HubScopedStore` (§6.7)	Flat `dict[str, ...]` keyed by the bare Rung-2 string — the silent-collision surface

Surface	Rung 1 (today)	Rung 2 (today)	Rung 3 (design)	Current leaf renderer
Tree nodes	none — `tree` carries no id today	none — inert per its own docstring	`hidden_id` once a node has one	N/A — no selection model yet

For every one of these, the same two renderings apply, from the one `LuxAddress`: the **hidden ImGui id** (`hidden_id` — never the label, never the bare Rung-2 string alone) and the **visible title** (`title(...)` — hostname-anchored; the Hub rung elided while it is unambiguous, the connection rung always shown on an owned frame, per DES-100). One Hub, one owning session: `"lux :: Vox"`. Two sessions sharing a repo, one Hub: `"lux (2) :: Vox"`. Two Hubs: `"pembroke :: lux (2) :: Vox"`. A frame with no owning session reads its bare title, `"Vox"`.

A single Display-side component, `AddressBook` — composed wherever `MenuReplica` and `SceneReplica` already live — is the one place that: tracks the live set of connected `HubId`s and, per Hub, the live set of connections; computes each rung's **ambiguity** (a pure cardinality test) once per frame or menu build, the same cadence `MenuReplica.menu_model()` already rebuilds every frame; and, once a rung *is* shown, reuses DES-064's collision-numbering rule and DES-067's `(repo, session_pid)` grouping to compute that rung's own **label text**. Ambiguity and labeling are two different jobs on two different cadences: ambiguity decides *whether* a rung shows at all; DES-064/067 numbering decides *what* a shown rung's label reads as. `AddressBook` exposes exactly one construction path, `address_for(hub, connection_key, connection_label, leaf_key, leaf_label) -> LuxAddress`; no leaf renderer may construct a `MenuItem`, a frame title, a scene entry, or a tree-node row directly from a bare label ever again.

6.7 Aggregated Storage: `HubScopedStore`

An aggregated store is an object, not a dictionary passed between functions. `FrameBook`'s `_scene_to_frame` and `_scene_to_owner`, and `MenuReplica`'s `_callback_menus`, are today flat `dict[str, ...]` mappings spanning every connection the Display has ever seen — a primitive collection standing in for a domain type, with the keying and purge logic living in whichever caller happens to touch the dict rather than on the collection itself. That is exactly what PY-OO-5 (behavior lives with its data) and PY-IC-1 (composition over primitives) in `punt-labs/.claude/rules` forbid, and what `punt-kit/standards/oo.md` names as the same mistake at the language-agnostic level. **This design specifies the replacement as a class, not a shape left to the implementer:** `HubScopedKey` is the key type, and `HubScopedStore` is the container every aggregator composes in place of its bare dict.

```
@final
@dataclass(frozen=True, slots=True)
class HubScopedKey:
    """An aggregated store's real key -- a Hub's own HubId, paired
    with the Rung-2 id that Hub already minted
    (ConnectionScopedId.compose's result, or
    CallbackInvocation.menu_id). Composed as a value type, never
    concatenated into one string and never a bare tuple: two
    independently-meaningful fields, kept as two fields, the same
    argument HubId itself makes for keeping hostname and pid apart.
    Carries no label -- LuxAddress carries this identical pair again
    for display, but a store's key answers "is this the same slot,"
    never "what should this be called." """
    hub: HubId
    local: str


@final
class HubScopedStore[V]:
    """The one aggregated-storage shape every Display-side collection
    that spans more than one Hub composes, in place of a bare dict.
    Owns its own keying discipline -- a caller never reaches past this
    class into a bare mapping -- and exposes only the operations an
    aggregator actually needs: assign, look up, enumerate one Hub's own
```

```
    entries, and retire one Hub's entries either wholesale (disconnect)
    or selectively (a manifest naming which of that Hub's own entries
    still exist). What structure backs this class is this class's own
    business and nobody else's; that encapsulation -- not any specific
    backing structure -- is the design-time commitment."""

    def __init__(self) -> None:
        self._entries: dict[HubScopedKey, V] = {}

    def put(self, key: HubScopedKey, value: V) -> None:
        """Assign one entry. Two entries collide only when they share
        the identical (hub, local) pair -- never across Hubs, by
        construction of the key type."""
        self._entries[key] = value

    def get(self, key: HubScopedKey) -> V | None:
        """Look up one entry. None answers a genuine "not present" --
        there is no discriminated state hiding behind it."""
        return self._entries.get(key)

    def for_hub(self, hub: HubId) -> Iterator[tuple[str, V]]:
        """Every entry a given Hub currently owns, local id and value
        -- the routing primitive a scene-less interaction or a menu
        click resolves its one target Hub through."""
        for key, value in self._entries.items():
            if key.hub == hub:
                yield key.local, value

    def purge_hub_not_in(
        self, hub: HubId, still_named: frozenset[str]
    ) -> list[V]:
        """Drop every entry this Hub owns whose local id is absent
        from ``still_named`` -- the manifest-purge operation, scoped to
        one Hub's own entries by construction, so a second live Hub's
        entries are never candidates."""
        stale = [
            key
            for key in self._entries
            if key.hub == hub and key.local not in still_named
        ]
        return [self._entries.pop(key) for key in stale]

    def drop_hub(self, hub: HubId) -> list[V]:
        """Retire every entry a Hub owned, on disconnect."""
        stale = [key for key in self._entries if key.hub == hub]
        return [self._entries.pop(key) for key in stale]
```

`FrameBook` composes two `HubScopedStore` instances in place of `_scene_to_frame` and `_scene_to_owner`; `MenuReplica` composes one in place of `_callback_menus`; the eventual `TreeSelectionModel` (`lux-kob7`) composes its own from the start, Rung-3-ready. No aggregator keeps a bare `dict[str, ...]` alongside its `HubScopedStore` as a second, competing source of truth for the same collection.

6.8 Multi-Hub Topology

Operator ruling (2026-09-04): cross-host is in scope now. One Display aggregates Hubs across multiple machines, not only multiple `luxd` processes on one host. The addressing model above — rungs, `LuxAddress`, `AddressBook`, per-store keying — is transport-agnostic and applies identically whether a Hub reaches the Display over a Unix socket or a network socket. How a remote Hub establishes that connection in the first place is a separate transport-and-trust design, §7.

6.8.1 What Already Works, Unmodified

The Hub-to-Display leg is, today, a Unix domain socket (mode `0700`) — the Display is already a multi-connection server, accepting an arbitrary number of client fds. Several pieces of the multi-Hub story are already built, for a reason unrelated to multi-Hub, and survive a same-host-to-network transport change unmodified because each was already written against "a connection" in the abstract:

- **Per-connection scene ownership.** `FrameBook.scene_to_owner` already maps `scene_id` to an fd, and interaction delivery already routes a scene-bearing interaction to its owning fd, not to whichever connection happens to be "the" Hub. This path is multi-Hub-safe today, unmodified, because it was already written per-connection for DES-068's reconnect/purge story.

Preemption is a required change, not something already correct. `SocketListener.hub_fd_for(name)` and its stale-hub eviction look up and evict a connection sharing the identical *declared name* — `ConnectMessage.name`, which this design keeps meaning "what a human calls this connection," never a per-process identity. Today every Hub process declares the identical hardcoded name, so single-Hub preemption reads correctly by accident. Once real, distinct `HubId`s exist, name-keyed preemption is unsafe: two independently-legitimate Hubs that happen to share a declared `name` would preempt each other, and a reconnecting Hub could evict an unrelated live one. **Resolution (this design, coordinated with §7): preemption must be re-keyed onto `HubId`, never `name`.** This is a required implementation step, listed as a shared bead in the increment-of-work document, not a property the current code already has.

6.8.2 What Must Change

1. **A Hub declares a real `HubId`, not a constant.** `ClientRegistry.get()` constructs its `DisplayLink` with a hardcoded name today; that becomes `HubId.current().wire_token`, carried on the dedicated `hub_id` field.

2. **Every Display-side store keyed by a Rung-2 string becomes a `HubScopedStore`.** `FrameBook`'s scene/frame maps and `MenuReplica`'s callback/agent-menu tuples — each currently a flat `dict[str, ...]` spanning every connection — are replaced by `HubScopedStore` instances (§6.7), keyed by `HubScopedKey` rather than a bare Rung-2 string. This is the design-time commitment, stated as a type, not left open as an implementation choice: no aggregated store may be a raw dict keyed by a bare Rung-2 string, or by anything less than a `HubScopedKey`, once more than one Hub can write to it.

3. **Scene-less interaction dispatch must stop broadcasting.** A menu-bar or World-panel click carries no `scene_id` and today broadcasts to every connected client, on the assumption that exactly one Hub is listening. With more than one Hub connected, a broadcast delivers every Hub's menu click to every Hub, and because `CallbackInvocation`'s connection id is only unique within the originating Hub, a stray click is a live misrouting risk, not merely wasted work. Once callback menus are stored per-`HubId` (point 2), the Display already knows which Hub contributed a given `CallbackInvocation` at menu-composition time; that `HubId` threads through to interaction delivery so it names its one target fd, and the broadcast fallback is dropped for menu-sourced events.

4. **Manifest purge is a required change, not something already correct.** `SceneReplica.scenes_to_purge` disowns any scene that is neither owned by the identifying fd nor named in the sending manifest — a rule written to sweep an orphaned scene left behind by a *prior* connection's death, since that orphan's owner is never the identifying fd either. Under real multi-Hub, the identical predicate just as readily disowns a *second, live* Hub's own scenes: Hub B's manifest names only Hub B's scenes, so every scene Hub A owns is, from Hub B's manifest handling's point of view, "neither owned by the identifying fd nor named in the manifest" — exactly the purge condition, and exactly a cross-Hub data-loss hazard, not the orphan-sweep it was written for. **Resolution: the purge predicate must be scoped by the sending Hub's own `HubId`, so a manifest disowns only scenes that Hub itself previously owned and no longer names, never another live Hub's.** This is a required implementation step, coordinated with point 2's per-Hub-keyed storage and listed as `W14` in the increment-of-work document, not a property the current code already has.

5. **Connect, same-host: no new logic.** A new same-host Hub process opens a new socket connection and sends its own `ConnectMessage`; the accept loop requires no change. Connect, cross-host, is

out of this section's scope — §7. Once a remote Hub's connection is established and authenticated there, points 1–3 and the per-surface treatment above apply identically; there is no second version of the addressing model for the cross-host case.

6. **Disconnect** needs no new logic beyond dropping the departed `HubId` from `AddressBook`'s live set, so a solo survivor's rung re-elides. Every other disconnect concern — per-fd scene reaping, lease sweep — is already fd-scoped.

7. **Elision churn under connect/disconnect is an accepted trade-off**, not a defect: because ambiguity is recomputed live, a title that was bare a moment ago can gain a `hub ::` prefix the instant an unrelated second Hub connects, and lose it again the instant that Hub disconnects. `LuxAddress.title()` is always an accurate answer for the population that exists right now; a mitigation (a sticky prefix, a transient highlight) is complexity purchased for a case the operator's own scope note treats as simple.

6.8.3 Discovery, Same-Host

There is no Hub registry, no discovery protocol, no broadcast, and no new authentication handshake for the same-host case. A Hub connects to the Display's already-published Unix socket path exactly as it does today. "Discovery" is unlocking a door that is already open: a second `luxd` process finds and connects to the identical socket the first one did, because nothing about the socket path is Hub-specific. Discovery and connection establishment for the cross-host case are delegated to §7.

6.9 Governing Invariant

> For any item the Display renders as part of a collection it aggregates — a menu entry, a frame or window, a scene, a tree node — that item's storage key and its ImGui widget id both derive from the item's full `LuxAddress` (hub, connection, leaf). Neither may derive from a bare human label, nor from a Rung-2-or-lower id alone, once more than one Hub can contribute to that collection.

This is checkable and structural rather than behavioral: it becomes a type discipline once `HubScopedKey` is the actual key type of every aggregated store, composed inside a `HubScopedStore` (§6.7) rather than passed around as a bare `dict`. A store typed `dict[str, Frame]` for something more than one Hub can populate is the violation, visible at the type signature — not merely at review time. Concretely, one regression test is the load-bearing check: two synthetic Hub connections independently producing colliding Rung-2 strings must produce two distinct entries in the Display's storage, never one clobbering the other.

Not a z-spec candidate. This invariant is a naming/keying discipline, not a concurrency property: there is no shared mutable resource two threads race over here, since each Hub's fd is handled by the existing single-threaded socket-callback dispatch, serially. A structural test plus a code-level check that every aggregated store's key type carries `HubId` is the right-sized verification.

6.10 Reconciliation with Existing Design

DES-088 (visibility is the Display's own axis) is orthogonal: `LuxAddress` answers "which storage slot, which widget id"; `FrameVisibility` answers "is it painted." DES-064/DES-067 (Clients-menu grouping and `(n)` numbering) are reused for labeling, not for elision — the two are different jobs, kept distinct above. DES-086 (scenes and frames cannot alias across connections) is unchanged at Rung 2; this design extends the identical discipline one layer up. DES-058 (`CallbackInvocation`) is unchanged and wrapped, not altered — a click still resolves against the originating Hub's own registry, which is exactly why misrouting a stray broadcast to the wrong Hub is a real risk this design closes.

7 Cross-Host Transport and Trust

> *Design status: DES-090 in `DESIGN.md`, Status: ACCEPTED — ratified by the operator 2026-09-05. Companion to §6 (DES-089), which states the five-point contract this design satisfies. Not yet implemented; the bead map and dependency order live in `docs/architecture/multi-hub-addressing-work.md`.*

Today the Hub-to-Display leg is a single mechanism doing two jobs at once: `AF_UNIX`, mode `0700`, one well-known path per user. The socket's filesystem permission *is* the entire trust model — anything that can open the path is, by construction, a process the same OS user already controls. §8's loopback policy states the analogous refusal explicitly for luxd's other leg (MCP/REST); the Hub-to-Display leg has never had an equivalent policy, because it has never needed one. Cross-host is exactly what gives it a reason to: the operator ruled (2026-09-04) that a single user's Display may aggregate Hubs running on other machines, now, not deferred, which breaks the one assumption the whole trust model rests on. A network socket has no equivalent free lunch — anyone who can route a packet to the Display's port can attempt to speak the Hub protocol to it.

Scope boundary. This design owns the network transport, who initiates a connection and how the endpoint is known, the authentication and trust mechanism, and how connect/reconnect/disconnect map onto the Display's existing fd-scoped ownership and preemption machinery. It does *not* own the addressing model (§6), and it does not own DES-086's connection-scoped read/write security boundary — it extends that boundary to a new transport, it does not alter it.

One direction only. The Hub is, in both the same-host and cross-host cases, the connecting party — it dials the Display, never the reverse.

7.1 Threat Model

Adversary. Lux's cross-host case is a single user's own multiple machines — a laptop, a desktop, a home server — not a multi-tenant service with mutually distrusting principals. The realistic adversary is someone or something on the network path who is *not* that user: another device on the same LAN or Wi-Fi, a compromised machine elsewhere on the network, or an opportunistic scanner. It is explicitly *not* a threat model against the user's own other machines — if the user's own laptop is compromised, no transport policy protects the Display from it, because the laptop already holds legitimate credentials. This is device-pairing for one person's own hardware, closer to SSH `known_hosts` or a personal CA than to a public PKI or an OAuth tenant model.

#	Can the adversary...	Without valid credentials	With a stolen/leaked credential
T1	Connect and speak the Hub protocol at all	**No** — TLS handshake fails closed before any application byte is read	Yes — the credential is the authorization
T2	Read another Hub's or the user's scene/menu content over the wire	No — TLS provides confidentiality; a passive eavesdropper sees ciphertext only	N/A — this axis is about eavesdropping, not impersonation
T3	Inject content attributed to a real, already-connected Hub	No — cannot present that Hub's private key	No — a stolen credential impersonates only the credential's own Hub
T4	Cause a collision in the Display's per-`HubId` stores by claiming an already-live hostname	No — hostname is cert-verified, not self-reported (§7.2)	Only the same machine the credential was issued to
T5	Bypass DES-086's per-connection read/write scoping once connected as a legitimate Hub	No — DES-086 is transport-agnostic	No — credential theft grants that Hub's own scope only
T6	Downgrade the connection to an unauthenticated or unencrypted one	No — the cross-host listener never accepts `kind="test"` and never falls back to plaintext	N/A

Consequences of credential loss. A stolen private key for one enrolled machine lets the attacker act as that machine's Hub — read and write exactly the content the real Hub would have, until the credential is revoked. This is symmetric with SSH: a stolen host key compromises that host's identity, not the whole fleet's. This design accepts coarse, whole-CA revocation as a proportionate trade-off for a personal, small-N deployment (§7.9).

Out of scope, explicitly. Denial of service beyond what TCP/TLS already provide; a malicious already-enrolled Hub process beyond DES-086's scoping (it never gains cross-Hub reach, but is fully trusted for its own scope, same as today); and physical or OS-level compromise of an enrolled machine.

7.2 Resolving the Trust Fork: `HubId.hostname` Is Transport-Verified

§6.4 flags, without resolving, whether `HubId.hostname` stays a transport-verified network name (option **a**) or becomes a cosmetic label beside a new opaque, transport-assigned uniqueness key — a certificate identity, an mTLS SPIFFE-style id, a connection-scoped token (option **b**). **Decision: option (a).** `HubId` keeps exactly the two fields already specified — no third field.

The real question is not which option is more defensive, it is what "verified" means if the transport authenticates the connection but not the field. If the transport authenticates the TLS session and then passes through whatever `hostname` string the Hub self-reports in `hub_id` unchecked, `HubId` is not actually verified — it is merely carried over a verified pipe. A misconfigured Hub (no malice required) could self-report a `hostname` colliding with a different, legitimately distinct Hub's already-live entry, reintroducing the silent-collision failure mode §6 exists to close, just moved past the transport boundary.

Option (a) closes the gap directly: the transport does not trust the self-reported `hostname` half of `hub_id` at all. It derives the verified hostname from the mTLS peer certificate's own Subject Alternative Name (bound to that certificate at enrollment, §7.8) and **rejects the connection outright if the self-reported hostname does not match the certificate's SAN.** `pid` is passed through as self-reported, unverified, exactly as `HubId`'s own docstring already treats it — it only ever breaks a tie within one already-identified host.

Option (b)'s own case for itself — "this hostname string is unique and genuine" is a harder claim to verify than "this credential is unique and genuine" — collapses once the hostname is derived from the certificate's SAN: verifying the credential *is* verifying the hostname, because enrollment is what attached the name to the credential in the first place. Option (b)'s cost — a third `HubId` field and a real/cosmetic split — buys nothing option (a) does not already deliver. **Consequence for `HubId`:** none beyond what §6.4 already specifies. **Consequence for the wire protocol:** none — `hub_id` still carries the full `HubId.wire_token`; cross-host adds a rejection rule at the transport boundary, not a new field.

7.3 Client Host: Observed on the Transport

The same principle covers the other host a person reads in the frame `i` popup and the Clients-menu Details window (the DES-099 amendment). **Client host** is the address the Hub *observed* on the connection's transport, never a value the client declares: `ClientIdentity` carries no host field and refuses unknown fields, so a declaration has nowhere to go. One reader, `PeerHost`, takes Starlette's socket peer (`client.host`) at each front door — REST, the WebSocket listen leg, and MCP streamable HTTP (bound per request beside the session key and released with it) — and the address reaches `ClientSession` in the same write as the identity. luxd builds its uvicorn config with `proxy_headers=False`: uvicorn otherwise rewrites the client address from an `X-Forwarded-For` header sent by any loopback caller, turning a declaration into the "observed" host. A local client reads `127.0.0.1` or `::1`; a client seen from several hosts under one identity (DES-057) lists them in first-seen order; a host the Hub did not observe reads `unknown`, never a defaulted loopback. **Hub host** is the Hub's own `HubId.hostname`, resolved once on first use and kept, so it is the same name the Display verified. Both ride the existing owner facts and the Details scene; no new message and no reverse DNS.

7.4 Transport Specification

Option	Confidentiality + integrity	Authentication	New moving parts	Verdict
TLS 1.3, mutual auth (client + server certs)	Built in	Built in — one handshake does both	One cert/key pair per machine; a trust anchor (§7.5)	**Recommended**
Plain TCP + application-level HMAC/token	None on the wire	Authenticates only after content already crossed the wire	Token issuance/rotation, plus still need TLS for confidentiality	Rejected
Server-only TLS + bearer token	Built in, one direction	Server authenticated; Hub authenticated only by the token	Token issuance/rotation, TOFU cert pinning	Rejected
SSH port-forward/tunnel	Built in	Built in	A dependency on the system SSH client/agent and a supervised tunnel process	Rejected
Public CA (Let's Encrypt-style)	Built in	Built in	Requires public DNS + inbound internet exposure for domain validation	Rejected

Recommendation: TLS 1.3, mutual authentication, against a trusted certificate authority. One mechanism supplies both confidentiality/integrity (T2) and authentication (T1, T3, T4) from a single, already-hardened protocol — Lux does not implement its own cryptography; it configures `ssl.SSLContext` with `verify_mode=ssl.CERT_REQUIRED` on both ends and `minimum_version=ssl.TLSVersion.TLSv1_3`, and everything else is the standard library's and OpenSSL's problem. **Who operates that certificate authority is a separate, pluggable decision, not a property of the mTLS mechanism itself** — §7.5, below, defines the two supported trust-anchor providers. The new code this design adds is a certificate-issuance/enrollment path and a socket-listener change, not a hand-rolled auth protocol.

Session resumption is left enabled on both sides, deliberately not hardened away: under TLS 1.3 a resumption ticket is cryptographically bound to the original full handshake — including the mutual certificate exchange — so a resumed connection carries the same verified identity, not a fresh or weaker claim. Python's `ssl` module exposes no public API for 0-RTT/early data, the one property of session resumption with a known replay weakness, so that risk does not arise here.

Why the rejected options lose. Plain TCP plus a token authenticates the holder but does nothing for the wire itself, and once TLS is added back for confidentiality the token becomes pure overhead — a second mechanism doing a job TLS already does more completely. Server-only TLS plus a bearer token is ordinary HTTPS-with-an-API-key and fails T3 more subtly: the token is a single shared secret, so revoking one compromised machine means rotating the token everywhere, and `HubId.hostname` would have nothing cryptographic to derive from — reopening the trust fork §7.2 just closed. SSH tunneling solves the problem well but makes the transport depend on an external program lux does not control, trading a library-level TLS configuration for a process-management problem; nothing prevents an operator from running Lux over an SSH tunnel or a VPN mesh instead, since the underlying wire protocol is transport-agnostic, but lux does not build that integration itself. A public CA needs public DNS and inbound internet exposure for domain validation, the wrong shape for a LAN/VPN-scoped personal tool — this row means a publicly-trusted, domain-validated web-PKI CA specifically, not a managed *private* CA an adopter operates for itself; §7.5 and §7.9 keep the two distinct.

7.5 Trust Anchor Providers: Pluggable, Not Fixed

The mTLS mechanism does not care who signs the certificate. It authenticates a Hub from its client certificate and derives the verified `HubId.hostname` from that certificate's own Subject Alternative Name (§7.2). Nothing in that verification rule, in the four invariants (§7.10), or in the T1–T6 threat model above depends on whose private key signed the leaf — an AWS-issued leaf is trusted by the identical SAN-verification rule as a personally-signed one. Only two things vary by provider: the trust anchor the Display verifies a leaf certificate's chain against, and the enrollment path by which a Hub obtains a correctly-SAN'd leaf in the first place.

The provider interface, stated minimally, is two obligations:

1. **Trust anchor.** The root (and any intermediate) certificate(s) the Display loads into its `ssl.SSLContext` as the verification set for an incoming client certificate. Swapping providers means swapping what this set contains; nothing else in §7.2's verification path changes.

2. **Enrollment path.** The procedure that, given a CSR whose SAN already names the requesting machine's hostname, signs it into a leaf certificate whose chain terminates at the trust anchor above. §7.8 specifies the enrollment mechanics for each provider; only the identity of the signer and the issuance channel differ.

Implementation (W7, `src/punt_lux/punt\_lux.trust`): the private-key/CSR half of enrollment is provider-*independent* and lives outside the protocol above it — generating a keypair and a self-signed CSR naming a hostname (`EnrollmentRequest.generate`) needs nothing from whichever CA will eventually sign it, and AWS Private CA's own `IssueCertificate` call takes a CSR exactly the same way. The `TrustAnchorProvider` protocol (a structural `Protocol`, no base class) therefore narrows to exactly the two obligations above, expressed as two methods — `trust_anchor() -> TrustAnchor` and `issue_leaf_certificate(csr) -> LeafCertificate` — and never a private key: `CertificateAuthority.sign_csr` rejects a CSR whose self-signature does not verify before issuing, and `EnrollmentRequest.complete` confirms the returned leaf's public key matches the request's own before accepting it as an `EnrolledIdentity`. `PersonalCaProvider` (Provider 1, below) is the first implementation; W15's AWS Private CA provider satisfies the identical two methods. A leaf's `AuthorityKeyIdentifier` must chain to its issuer's `SubjectKeyIdentifier` for OpenSSL 3.x's default verification policy to accept it (RFC 5280 §4.2.1.1) — both CA creation and leaf issuance set these extensions; their absence surfaces as *"Missing Authority Key Identifier"* at handshake time, not at signing time, so it is easy to omit and only discover under a real `ssl.SSLContext` handshake, which is why the W7 test suite drives one over TCP loopback rather than asserting on the `cryptography` objects alone.

Any implementation satisfying both obligations is a valid trust-anchor provider. The mTLS transport code itself never branches on which one is configured — it loads `ssl.SSLContext`'s verification set and serves whatever leaf the active provider issued, and the handshake, the SAN check, and every T1–T6 row hold identically past that point. Which provider is active is a per-Display configuration choice made once, at cross-host-enable time, not a per-connection or per-Hub choice — every Hub a given Display enrolls trusts the same anchor.

Two providers ship with this design:

- **Provider 1 (default): self-managed personal CA.** Exactly what §7.8 specifies: free, the CA's private key never leaves the Display's own machine, enrollment is a manual offline CSR exchange, and revocation is coarse — whole-CA regeneration and re-enrollment (§7.9). Zero external dependency, single-machine key custody: the correct default for the deployment this design targets, a handful of machines one person owns.

- **Provider 2 (optional): AWS Private CA (managed).** The Display trusts the AWS Private CA root, or an intermediate issued under it, instead of a locally-generated root. A Hub obtains its leaf certificate through ACM Private CA issuance — an API call the operator can gate with IAM policy — rather than an offline CSR exchange, and revocation is proper CRL/OCSP rather than whole-CA regeneration. The trade-off is explicit, not hidden: paid and managed, with an AWS account and IAM as a new external dependency, in exchange for issuance and revocation that scale cleanly to an organization with many machines and users. This suits an enterprise adopter, not the single-user default this design otherwise targets; consult AWS Private CA's current pricing before adopting it — it is an enterprise-tier, monthly-per-CA-plus-per-certificate cost, and this design pins no specific figure that would go stale.

This is not the rejected public CA. §7.9 and the Transport Specification table above reject a *public*, domain-validated web-PKI CA (Let's Encrypt-style) for needing public DNS and inbound internet exposure. AWS Private CA is a managed *private* CA — its root is never in any browser or OS trust store, exactly like Provider 1's self-signed root, and it requires no public DNS or inbound exposure. The two are different animals answering different questions; §7.9 keeps them distinct explicitly.

7.6 Coexistence with the Local Fast Path

The `AF_UNIX` leg is unchanged and stays the default. Cross-host is additive and opt-in: the Display gains a second listening socket, bound only when the operator explicitly enables it, mirroring the loopback policy's fail-closed default — no network listener exists until asked for, and an unconfigured bind stays loopback-only, so "enabled but misconfigured" degrades to "still local," never to "silently reachable from the network."

Both legs feed the same socket-listener machinery: `ssl.SSLSocket` is a subtype of `socket.socket`, so `select`, `recv`, and `send` behave identically once the TLS handshake has completed, and every fd-keyed dict in the socket server needs no change. The one change is in the accept path: it must poll a second listening socket and must not hand a freshly-accepted TLS connection to the ordinary accept path until the handshake — including client-certificate verification — has fully completed.

The handshake itself must never block the render thread. Accept and poll both already run synchronously inline on the single render thread, once per frame — the reason the socket listener already carries a per-frame deadline bounding every other network wait it performs. A blocking `do_handshake()` call has no such bound: a peer that completes the TCP handshake and then stalls or trickles its `ClientHello` would hang that call for as long as it likes, freezing rendering *and* the same-host `AF_UNIX` leg for every Hub, with no credential required — exactly T1's opportunistic scanner, and exactly the render-loop-freezing denial of service this threat model rules out. Fail closed, not hung.

The implementation must instead drive the TLS handshake **non-blocking, pumped through the existing accept/poll cadence, under a bounded per-connection deadline**, mirroring the existing per-frame deadline pattern rather than inventing a new one:

1. Wrap each newly-accepted `AF_INET`/`AF_INET6` socket in the server's `ssl.SSLContext` with `do_handshake_on_connect`, set it non-blocking, and hold it in a new pending-handshake set keyed by fd — kept separate from the client/reader dicts, so no fd reaches either before its handshake is verified complete. Record a deadline (a small constant, on the order of a few seconds) alongside the pending entry.

2. Each frame, the accept path calls `do_handshake()` once per pending connection. `ssl.SSLWantReadError`/`ssl.SSLWa` are the expected non-blocking retry signals; any other `ssl.SSLError` (untrusted CA, bad cert, protocol failure) closes the socket and drops it immediately.

3. A pending connection whose deadline has passed is closed and dropped unconditionally, mid-handshake if need be — the fail-closed backstop.

4. **Two distinct gates, not one.** *Gate 1:* once `do_handshake()` returns successfully, the fd leaves the pending-handshake set and enters the ordinary reader set, where it can exchange `ConnectMessage`/`ReadyMessage` like any other connection — TLS-verified, but not yet `HubId`-verified. *Gate 2:* only once the SAN/`hub_id` cross-check (§7.2) passes, after `ConnectMessage` has arrived, is the connection's content accepted and its `HubId` entered into `AddressBook`'s live set. A connection sits between the two gates — readable, but not yet content-trusted — for exactly the span of the connect sequence below; no `SceneMessage` or other content-bearing message is honored from it until Gate 2 passes (Invariant 1, §7.10).

This keeps the entire cross-host listener on the same single render thread as everything else in the socket layer — no new thread, no lock, no cross-thread handoff of any render-thread-owned state. A bounded background thread performing accept-plus-handshake off-thread was considered and rejected: it would be the first thread ever introduced into the socket listener's otherwise single-threaded, lock-free design, and the queue handoff becomes a second interleaving surface needing its own correctness argument, for a problem the existing per-frame-budget pattern already solves without one. Minimizing new concurrent machinery is the security argument here, not a style preference: fewer interleavings is less attack surface to reason about, model, and get wrong.

The wire protocol itself does not change. A Hub process's own connection code differs only in which socket family and `ssl` wrapping it uses to reach the Display, not in what it sends once connected.

7.7 Discovery and Connection Lifecycle

Endpoint discovery is pragmatic, with no registry. The Hub is the connecting party, and the instruction is not to over-engineer discovery. There is no Hub registry, no broadcast, no service-discovery protocol, and no rendezvous server. A remote Hub's machine holds a small, per-user config file naming the Display's `host:port` and the fingerprint of the trust anchor it should trust (§7.5) — the network equivalent of the Display's socket path being one well-known filesystem location per user. Populating that file is part of enrollment (§7.8), a one-time, explicit, user-driven step, not a live discovery protocol. If the Display's reachable address changes (a DHCP lease renews, a public IP rotates), the remote Hub's config goes stale until updated, or until superseded by whatever stable-addressing mechanism the user already relies on (a static LAN reservation, a VPN mesh, dynamic DNS) — Lux does not attempt to solve address stability itself.

Who initiates, and why that direction is kept. The Hub dials the Display in both cases, the same direction the `AF_UNIX` leg already uses. If the Display initiated connections to Hubs instead, it would need to discover Hub endpoints itself — reintroducing the registry ruled out above — and it would need Hub-side listeners bound and reachable, multiplying the number of network-facing accept loops from one to one per Hub. Keeping the Display as the sole listener keeps the network attack surface to exactly one opt-in, authenticated port, everywhere.

Connect, cross-host:

1. The Hub opens a TCP connection to the Display's configured `host:port`.
2. Both sides perform the TLS 1.3 handshake with mutual certificate verification. A connection whose peer cannot present a certificate signed by the trusted CA is rejected at this step — no application byte is ever read from it (T1).
3. Once the handshake completes, the Display sends `ReadyMessage` — the identical first frame the `AF_UNIX` leg sends today, unchanged.
4. The Hub responds with `ConnectMessage(kind="hub", hub_id=...)`, the identical second frame, unchanged.
5. The Display extracts the verified hostname from the peer certificate's SAN, compares it against the `hostname` half of the self-reported `hub_id`, and rejects (closes the connection, logs server-side) on any mismatch. On a match, `HubId` is now verified and enters `AddressBook`'s live set exactly as the same-host case does.
6. From this point forward, everything in §6's model applies identically to this connection and to a same-host one.

A cross-host connection never declares `kind="test"`. The test-kind backdoor exists for local development, and its safety today rests on the same `0700`-socket trust the rest of the `AF_UNIX` leg relies on. The cross-host listener refuses `kind="test"` outright — closing the connection rather than accepting a read-only backdoor whose entire safety argument does not carry across a network (T6).

Reconnect. A dropped TCP connection (network blip, process restart) reconnects through the identical connect sequence above. `hostname` is stable across any reconnect, including a full process restart, because the enrolled certificate lives on disk on the Hub's machine and does not change between runs. `pid` does not survive a genuine OS process restart — a crash-respawn gets a new pid, a property of `HubId`'s own field choice (§6.4), not a defect this design introduces. The same-host-duplicate path already handles it correctly (`pembroke`, `pembroke (2)`, re-eliding once the stale entry is reaped).

Preemption — resolved. §6's "What Already Works" subsection states the resolution: single-owner preemption must key on `HubId` — the value this design guarantees is both verified and hostname-stable across reconnects — never on `ConnectMessage.name`, which stays a human label that several Hub processes may legitimately share. For a single same-host Hub this was harmless, because there was only ever one Hub to preempt; once cross-host makes two independently-legitimate, simultaneously live Hubs possible, name-keyed preemption risks a reconnecting Hub on one machine evicting a live, unrelated Hub on a different machine that happens to share the same declared name. This is one coordination point between the two designs, resolved once rather than decided twice: whichever bead reworks preemption keys it on `HubId`, shared between §6's per-Hub-keyed-storage work and this design's own connection lifecycle. See the increment-of-work document for the owning bead.

Disconnect. No new mechanism. A dropped cross-host connection is, from the socket listener's point of view, an ordinary fd closing — the existing per-fd scene reaping, `AddressBook` live-set removal, and lease sweep apply exactly as §6 already specifies for the same-host case. Nothing about the TLS layer needs its own teardown path beyond closing the underlying socket, which tears down the TLS session with it.

7.8 Authentication and Enrollment

The trust anchor is pluggable; the verification rule it feeds is not. Both providers below produce the identical material §7.2 needs: a leaf certificate whose SAN is the enrolled machine's hostname, chaining to one trust anchor the Display verifies against. What differs is only where that anchor lives and how a Hub gets its leaf. **Provider 1 is the default** for every deployment this design targets, and the rest of this design (the threat model, the invariants, the connect sequence) is specified against it without further qualification. Provider 2 is an optional alternative, detailed in its own subsection below, for an adopter who already operates AWS infrastructure and wants managed issuance and revocation instead of self-managed key custody.

7.8.1 Provider 1 (Default): Self-Managed Personal CA

The trust root is the user, not a service. The person enrolling a second machine already has authenticated access to both machines — they are not proving their identity to Lux, they are telling Lux "these two machines are both mine." Enrollment is a manual, explicit, out-of-band act, not a live network protocol Lux implements and must then defend on its own. This mirrors SSH's own `known_hosts` model more than a corporate PKI's.

Mechanism.

1. **CA creation (once, on the Display's machine).** The first time cross-host is enabled, the Display generates a small personal Certificate Authority — a private key and self-signed root certificate — stored under `~/.punt-labs/lux/ca/` with the same discipline the socket directory already uses (`0700` directory, `0600` private key). This CA signs only Lux's own leaf certificates; it is never installed into the OS or browser trust store. The private key should additionally be passphrase-protected at rest, and the CA directory kept out of any dotfile-sync or cloud-backup tool the operator runs elsewhere.

2. **The Display's own leaf certificate.** The Display issues itself a leaf certificate (SAN = the hostname it will present as) signed by that CA, for the server side of the mTLS handshake.

3. **Per-machine enrollment (once per additional Hub machine).** On the machine that will run a Hub, the operator generates a local keypair and a certificate signing request naming that machine's own hostname as the SAN. The CSR is signed by the Display's CA — **the CA's private key never leaves the Display's machine.** The signing step is a manual, offline exchange: the operator copies the CSR to the Display however they already move files between their own machines, and copies the signed certificate back.

4. **Result.** Each enrolled machine holds its own private key and a certificate binding it to its own hostname, signed by one shared CA both sides already trust — exactly the material §7.2 needs.

Implementation (W7). `src/punt_lux/punt\_lux.trust.CertificateAuthority` implements steps 1–2 above (`create`/`save`/`load`, `issue_leaf` for the same-machine case); `src/punt_lux/punt\_lux.trust.EnrollmentRequest/src/punt_lux/EnrolledIdentity` implement step 3's two-machine round trip. Keys are ECDSA P-256 (`cryptography.hazmat.primitives.asymmetric.ec.SECP256R1`), the modern default curve for TLS leaf/CA material. **The passphrase-protection this section recommends is not yet implemented**: `KeyPair.to_pem` serializes with `serialization.NoEncryption()`, relying solely on the `0600` file-permission discipline stated above. A passphrase layer belongs at whichever surface first collects operator input for key material — plausibly the enrollment CLI a later bead adds, not the `punt_lux.trust` value classes themselves, which have no prompt to collect one from. This gap is deliberate-for-now, not silently dropped: tracked as follow-on scope, not closed by W7.

A friendlier enrollment flow — a short-lived pairing code exchanged over the LAN, closer to a device-pairing UX — is a real usability improvement over copy/paste CSR signing, but it is its own protocol

with its own threat model (a pairing code is a bearer credential with a race between generation and use) and is deliberately out of scope for this design, noted as a rejected-for-now alternative rather than a gap.

Certificate lifetime, expiry, and rotation. The CA's self-signed root is issued with a long validity (10 years), regenerated only on a full re-enrollment event, not on a schedule. Each leaf certificate is issued with a much shorter validity (1 year), bounding the damage window of a leaked leaf key without adding the operational cost of a leaked CA key. An expired leaf fails the TLS handshake with a certificate-verification error on both sides — correct, fail-closed behavior, no different in kind from any other handshake rejection. What is not yet solved is diagnosis: today's reconnect loop treats every connection failure alike, so an expired leaf looks identical to "Display unreachable" or "wrong endpoint configured." This design requires the implementation to close that diagnosability gap: the Display's server-side log must record the specific rejection reason (never surfaced to an unauthenticated peer, to avoid an oracle), and the Hub-side client, which does hold a legitimate identity, should surface a distinct, actionable message ("certificate expired — re-enroll this machine") rather than a generic connection-failure retry loop.

There is no automatic renewal in this design. A leaf's expiry is a fully manual event — the operator re-runs the enrollment flow for the one affected machine. For a handful of personally-owned machines, a yearly manual touch per machine is a proportionate cost; an automatic-renewal protocol would be new standing machinery to save an operation this design already keeps deliberately rare and manual for every other lifecycle event.

7.8.2 Provider 2 (Optional): AWS Private CA (Managed)

What AWS Private CA is, at design level. AWS Private CA (formerly ACM Private CA) is a managed private-certificate-authority service: it operates a root and, optionally, subordinate CAs on the operator's behalf, issues private X.509 certificates, supports CRL and OCSP for revocation, and gates issuance with IAM policy. It is mTLS-compatible in the ordinary sense — a leaf it issues is an X.509 certificate like any other and needs nothing special from Lux's `ssl.SSLContext` configuration beyond pointing the verification set at its root. This design states AWS Private CA's capabilities at this level deliberately; it does not pin a specific feature claim beyond what is stated here without independent verification against AWS's own current documentation before implementation.

Mechanism.

1. **CA creation (once, in the operator's AWS account).** The operator provisions a root, and optionally a subordinate, CA in AWS Private CA under their own AWS account and billing. Lux does not create or manage this AWS resource itself — provisioning is an out-of-band AWS console/CLI/infrastructure-as-code step the operator performs once, the same way Provider 1's CA creation is a local script the operator runs once, just against a different substrate.

2. **The Display's own leaf certificate.** The Display requests a leaf certificate from AWS Private CA (SAN = the hostname it presents as) via ACM Private CA issuance, and installs the issued certificate for the server side of the mTLS handshake. The code path that loads a server-side leaf into `ssl.SSLContext` does not change between providers — only where the leaf came from does.

3. **Per-machine enrollment (once per additional Hub machine).** On each machine that will run a Hub, the operator requests a leaf certificate from AWS Private CA naming that machine's own hostname as the SAN — an ACM Private CA issuance API call, gated by whatever IAM policy the operator attaches to the calling principal, rather than Provider 1's offline CSR exchange with the Display.

4. **Result.** Identical in shape to Provider 1's: each enrolled machine holds a private key and a SAN-bound leaf, chaining to one trust anchor — here, AWS Private CA's root or intermediate — both sides already trust, exactly the material §7.2 needs.

Revocation is the concrete capability difference. Provider 1 accepts coarse, whole-CA revocation as a proportionate trade-off for a personal, small-N deployment (§7.9). AWS Private CA supports proper per-certificate revocation via CRL and OCSP: revoking one compromised machine's leaf does not require regenerating the CA or re-enrolling every other machine. This is not a contradiction of §7.9's rejection of "a live CRL/OCSP responder" — that entry rejects *Lux running its own* CRL/OCSP infrastructure as disproportionate for a personal deployment. Provider 2 does not ask Lux to run one; it

uses AWS's already-running one. That is exactly why Provider 2 is scoped to an adopter who already carries an AWS/IAM dependency rather than being the default.

Cost and dependency, stated plainly. AWS Private CA is enterprise-tier pricing — see AWS Private CA's current pricing for figures this design deliberately does not pin, since they are AWS's to change; at the time of writing the shape is a monthly per-CA charge plus a per-certificate-issued charge. It also adds an AWS account and IAM as a new external dependency Provider 1 has none of. The trade this design states without hedging: Provider 2 buys IAM-gated issuance, proper revocation, and a path that scales cleanly to an organization with many machines and users, at the cost of money and an AWS dependency. It suits an enterprise adopter, not a single user running Lux on a laptop and a home server — which is why Provider 1 stays the default and Provider 2 stays opt-in.

7.9 Rejected Alternatives

- **Distribute the CA's private key to every machine.** Simpler than CSR signing, but multiplies the number of places the single most sensitive secret in this design lives — every enrolled machine becomes capable of impersonating any Hub, not just itself, failing T3 and T4 more broadly than a leaked leaf key does.
- **A pre-shared/derived token instead of mTLS.** Rejected because it needs TLS added back for confidentiality regardless, at which point mutual TLS supplies both properties from one mechanism, and because a shared token cannot give `HubId.hostname` anything cryptographic to anchor to.
- **Leveraging the org's ethos/GPG identity infrastructure.** Rejected for a mismatch of what is being identified: ethos identities are bound to a person or agent, while `HubId` needs to identify a machine — the same person's two laptops need two distinct, independently-revocable identities, not one shared identity authenticating both. Reusing ethos's GPG keys would either issue one key per machine anyway (a parallel, second PKI with none of this design's tooling) or authenticate by person and leave machine disambiguation to the unverified `hostname` field again.
- **Revocation via a live CRL/OCSP responder.** Rejected for this deployment's scale: running a CRL/OCSP responder is exactly the heavy infrastructure the operator ruled against, for a threat this design already scopes to a handful of machines one person owns. The accepted trade-off instead: revoking one compromised machine means regenerating the CA and re-enrolling every machine, an explicit, bounded, rare, fully manual operator action — not a hidden footgun, because nothing silently trusts a revoked key in the meantime.

Reconciliation: a managed private CA is not the rejected public CA. The Transport Specification table, above, rejects a *public*, domain-validated web-PKI CA (Let's Encrypt-style) for needing public DNS and inbound internet exposure for domain validation — a mismatch with a LAN/VPN-scoped personal tool, not a judgment against managed CAs in general. AWS Private CA (§7.8.2) is a managed *private* CA: its root is never installed into any browser or OS trust store, it needs no public DNS record and no inbound internet exposure, and it is trusted by Lux the same way Provider 1's self-signed root is — as an explicitly-configured, non-public trust anchor. It is listed here as a distinction, not a contradiction: this design still rejects the public/web-PKI option; it separately *accepts* a managed private CA as an optional, non-default provider (§7.5).

7.10 Invariants

Stated precisely, so the implementation and its tests have a checkable target:

1. **No content before verification.** No `SceneMessage`, callback-menu message, or manifest message is processed from a connection until that connection's `HubId` has been both transport-verified (TLS handshake with client-cert verification complete) and cross-checked (self-reported `hub_id` hostname matches the certificate SAN).
2. **At most one live connection per `HubId`.** Once preemption is re-keyed onto `HubId` (above), a second connection presenting the identical verified `HubId` evicts the first, never coexists with it — the same single-owner guarantee DES-068 already established for the (today, single) same-host Hub, generalized to N verified Hubs.

3. **Hostname verification is fail-closed.** A SAN/`hub_id` mismatch, an untrusted or expired certificate, or a `kind="test"` declaration on the cross-host listener all result in connection rejection, never a degraded-trust fallback.

4. **The `AF_UNIX` leg's trust argument is untouched.** Nothing in this design weakens, bypasses, or shares state with the local socket's `0700`-permission trust boundary.

All four invariants hold identically under either trust-anchor provider (§7.5). None of the four is stated in terms of who signed a certificate — they are stated in terms of the TLS handshake, the SAN/`hub_id` cross-check, and connection state, all of which are provider-agnostic by construction. An AWS-issued leaf satisfies Invariant 1 and Invariant 3 by the identical verification code path a self-signed leaf does; Invariant 2's `HubId`-keyed preemption and Invariant 4's untouched `AF_UNIX` boundary do not reference the certificate provider at all. Only certificate *provenance* and the *enrollment* flow that produced the leaf differ between providers — never the mechanism these invariants govern.

7.11 z-spec Assessment

Invariants 1 and 2 above are **z-spec-REQUIRED for the implementation phase.** This design does not model them now — flagging is the deliverable here, not the model:

- **Invariant 1** is a stateful-protocol safety property with a real interleaving. The connection's own state — TCP connected, TLS handshaking, TLS verified awaiting `ConnectMessage`, identified with `HubId` verified, receiving content — is exactly the shape a z-spec trigger names. The concrete race a model-check should exhibit and then exclude: a non-blocking accept path that adds a socket to the client set before its TLS handshake or its `ConnectMessage` has actually completed, letting a `SceneMessage` arrive against an unverified or not-yet-identified connection.

- **Invariant 2** is a lock/ownership discipline across a reconnect race, with multiple remote Hubs interleaving — the same shape DES-068's own single-owner preemption already needed a careful sequential argument for, generalized from one Hub to N concurrently-reconnecting Hubs. This is precisely the recurrence signal that calls for a formalized state machine rather than another empirical round.

Invariants 3 and 4 are boundary checks and a non-interference statement, not concurrency properties, and are verified by a rejection-path regression test per threat-model row with a "No" answer under "Without valid credentials" (T1, T3, T4, T6 — T2 and T5 are not rejection tests) and a code-level audit instead. The fidelity requirement, stated in advance: the eventual model for invariant 1 must reproduce the defect when the handshake-before-accept ordering is removed, and the model for invariant 2 must reproduce a double-owner state when preemption is re-keyed incorrectly (kept on `name`, with two same-named-but-different-`HubId` connections both live).

7.12 Reconciliation with Existing Design

DES-086 (scene inspection is a per-connection security boundary) is preserved verbatim, not weakened: it operates on Rung 2 (`ConnectionId`) — a connection reads and writes only content composed on its own `ConnectionId`. This design operates one layer below that: it decides whether a connection is allowed to exist and speak at all, and what `HubId` it is bound to once it does. An authenticated, verified remote Hub gains exactly the same DES-086 scoping a same-host Hub already has.

This design satisfies, point for point, the five requirements §6 states for the cross-host transport layer: authenticate before content (Invariants, above, and the connect sequence); the addressing layer performs no authentication of its own and trusts `HubId` once delivered; `HubId` stability across reconnects (the hostname-stable/pid-unstable distinction, stated precisely); the transport owns endpoint resolution and connection initiation; and `hub_id` frame ordering (step 4 of the connect sequence).

A finding against the current same-host code, surfaced because cross-host depends on it. The Display-side scene-message handler rejects a `SceneMessage` only when the sending fd has already declared `kind="test"`. An fd that has not yet sent any `ConnectMessage` at all — `kind_of(fd)` returns `None`, which is not equal to `"test"` — is not rejected today; its `SceneMessage` is processed and installed. This is harmless on the `AF_UNIX` leg only because the socket's `0700` permission already means an unidentified fd is still a same-user process. It is not harmless once a store is keyed by `HubId`: there is no `HubId` to key an unidentified connection's content under at all. **This design requires closing**

this gap as a prerequisite for cross-host, not an optional hardening: every content-bearing message handler must reject a message from any fd where `kind_of(fd) is None`, not only `kind_of(fd) == "test"`, uniformly across both transports. Listed first in the increment-of-work document's bead map.

8 Transport and Process

luxd runs one FastAPI application on one uvicorn port (default 8430). Two surfaces mount on that app: the MCP leg at `/mcp`, and the typed REST routers beside it. A `/health` route reports liveness and the live session count.

8.1 The MCP Leg: Streamable HTTP

luxd's MCP surface is streamable HTTP (`src/punt_lux/mcp\_transport.py`), the MCP SDK's current transport, mounted as an ASGI application at `/mcp` on the same FastAPI app as the REST routes. There is no separate WebSocket endpoint, and mcp-proxy no longer sits in lux's path: Claude Code connects to `http://127.0.0.1:8430/mcp` directly through its native HTTP MCP configuration.

Each MCP session has its own identity. `McpAsgiApp` (`src/punt_lux/mcp\_endpoint.py`) resolves the session identity on each request and refuses the reserved REST connection key. `SessionScopedServer` (`src/punt_lux/mcp\_session.py`) runs the Hub cleanup cascade when a session ends, and `session_cleanup.py` tears down that session's Hub-side menu items, subscriptions, and inbox; its scenes stay behind (see §5, Scenes Survive Their Session). A vanished client — a killed process, a dropped socket — is reaped 30 minutes after its last activity, so its Hub state cannot leak until restart. Session teardown is isolated: one session ending does not disturb another.

8.2 The Loopback Policy

luxd binds loopback by default and refuses a non-loopback `--host` at startup with a clear message, so the bind and the trust policy can never disagree. One `LoopbackTransportPolicy` (`src/punt_lux/transport\_policy.py`) is the single source of the loopback allowlist: it decides which hosts luxd starts on, and it projects the same allowlist into the SDK's transport security settings so the streamable-HTTP transport rejects a foreign `Host` header (DNS-rebinding) or `Origin` header (cross-site). Off-loopback access needs authentication that is not built yet; the multi-machine future in `target.md` arrives together with a bearer token and a bind-derived origin policy — a different leg (MCP/REST), not the Hub–Display socket, whose own off-loopback story is mTLS (§7, DES-090), not a bearer token.

8.3 The Display Socket Is Hub-Internal

The Display listens on a Unix domain socket, and luxd is its only client. Nothing else in the package opens that socket. `DisplayLink` (`src/punt_lux/domain/hub/display\_link.py`) is the class that opens it, and it is constructed in exactly one place: `ClientRegistry` in `src/punt_lux/domain/hub/clients.py`, which holds luxd's one lazy connection and its reconnect policy.

An AST guard enforces this (`tests/domain/test_display_socket_internal.py`). The guard parses every module in the package and fails if any module outside the allowed set — `src/punt_lux/domain/hub/display\_link.py` and `src/punt_lux/domain/hub/clients.py` — imports or references `DisplayLink`. Detection is by parse tree, not by regex, so every spelling (an aliased import, a multi-line parenthesized import, a whole-module import, a bare or attribute reference) is caught at once, while a mention in a docstring or comment is never mistaken for use. A new reach-around cannot slip back in through a module an enumerated list would have forgotten.

8.4 Display Socket Discovery and Auto-Spawn

The Display's socket path resolves in priority order (`src/punt_lux/paths.py`):

1. `$LUX_SOCKET`.
2. `$XDG_RUNTIME_DIR/lux/display.sock`.

3. `/tmp/lux-$USER/display.sock` (fallback, chosen to stay within macOS's ~104-character `AF_UNIX` path limit).

A PID file is written at `{socket_path}.pid` on startup, and luxd verifies liveness via `os.kill(pid, 0)` before connecting. `DisplayPaths.ensure()` spawns the Display if it is not running, detached from the caller's process group, and polls socket existence and PID liveness every 100 ms until ready or a 5 s timeout. When a send to the Display fails with an `OSError`, the replicator drops the stale client so the next send reconnects.

8.5 The Display Render Loop

The Display runs an ImGui render loop via `hello_imgui.run()`. Each frame, the `RenderLoop._on_frame()` callback runs four non-blocking phases: accept new socket connections, poll the connection for messages and dispatch each by type, render the active scenes and frames, and flush any queued interaction messages back to the Hub. A frame completes in microseconds when idle; `hello_imgui.fps_idling.fps_idle = 30.0` caps the idle rate, and GLFW vsync paces active frames.

Table filtering runs in the render loop with zero Hub round-trips. `TableRenderer` draws the search and combo filter controls, applies AND logic across active filters (case-insensitive substring for search, exact match for combos), resets pagination to page zero on a filter change, and renders a two-column detail grid for the selected row.

9 Client Surfaces

Every surface is a thin adapter over the operations facade (§2.2). An MCP tool parses its arguments, calls one operation, and formats the result; it holds no validation, no scene construction, and no display round-trips.

9.1 MCP Tools

The display-facing tools live in `src/punt_lux/tools/tools.py` and `src/punt_lux/tools/composite\_tools.py`; the session pub-sub tools live in `src/punt_lux/tools/subscribe\_tools.py`.

Tool	Operation and purpose
Scenes	
`show`	`render` — install a scene from element dicts.
`show_table`	`render_table` — compose a filterable table tree and delegate to `render`.
`show_dashboard`	`render_dashboard` — compose a metrics, charts, and table tree and delegate to `render`.
`update`	`update` — apply patches by element ID.
`clear`	`clear` — remove the caller's scenes.
Menus	
`set_menu`	`set_menu` — replace the Hub-owned menu bar.
`register_tool`	`register_menu_item` — add a tool item for the caller's session.
`list_menus`	`list_menus` — read the menu bar.
Introspection (Hub-authoritative)	
`inspect_scene`	`inspect_scene` — a scene's element tree from the store, with each element's `render_path` (constant `"abc"`) and `resolved_props`.
`list_scenes`	`list_scenes` — every live scene and frame from the store.
`list_clients`	`list_clients` — the Hub's sessions and their scopes.
Display facts (proxied)	

Tool	Operation and purpose
`get_display_info`	`get_display_info` — backend, geometry, FPS, PID, uptime.
`get_window_settings`	`get_window_settings` — opacity, font scale, decoration, idle FPS.
`set_window_settings`	`set_window_settings`.
`get_theme`	`get_theme` — the active theme and the available themes.
`set_theme`	`set_theme`.
`set_frame_state`	`set_frame_state` — minimize or restore a frame.
`list_recent_events`	`list_recent_events` — the display's recent interactions.
`list_errors`	`list_errors` — the display's recent errors.
`screenshot`	`screenshot` — refuses with `rejected`: framebuffer capture is unsolved below the message layer (DES-028).
`ping`	`ping` — display liveness probe.
Per-repo config	
`display_mode`	`read_display_mode`.
`set_display_mode`	`write_display_mode`.
Session pub-sub	
`subscribe`	`subscribe` — subscribe the session to a topic in its own scope.
`unsubscribe`	`unsubscribe`.
`publish`	`publish` — fan a business event to the session's subscribers.
`recv`	`receive` — take the next session-scoped business event. This is not the display interaction stream.

An introspection tool reads Hub-authoritative state: `inspect_scene`, `list_scenes`, and `list_clients` ask the Hub, not the Display. A display-fact tool — a theme, a window setting, a framebuffer, the display's own ring buffers — proxies a read or write over luxd's one connection. The Hub cannot own an ImGui theme or a GPU backend string, so those stay display facts; the caller still reaches them the same way, through a Hub operation. Menu state is the deliberate exception: a menu is interface the agent submitted, so the Hub owns it and the replicator pushes it like any scene.

9.2 REST Routes

The REST surface (`src/punt_lux/rest/`) binds each route body to a request model, calls one operation, and returns the result model; FastAPI derives the OpenAPI schema from the model, so there is no second schema to maintain.

Method and path	Operation	Result
PUT /scenes/{id}	render	SceneShown | OpError
PATCH /scenes/{id}	update	SceneShown | OpError
DELETE /scenes	clear	Cleared
GET /scenes	list_scenes	SceneList
GET /scenes/{id}	inspect_scene	SceneInspection | OpError
GET /clients	list_clients	ClientList
GET /menus	list_menus	MenuList
PUT /menus	set_menu	Ok | OpError
POST /menus/items	register_menu_item	Ok | OpError
GET /events	list_recent_events	RecentEvents
GET /errors	list_errors	RecentErrors
GET /display	get_display_info	DisplayInfo | OpError
GET,PUT /display/theme	get/set_theme	ThemeState | OpError
GET,PATCH /display/window	get/set_window_settings	WindowSettings | OpError
PATCH /display/frames/{id}	set_frame_state	Ok | OpError
GET /display/screenshot	screenshot	Screenshot | OpError
GET /display/ping	ping	Pong | OpError
GET,PUT /display-mode	read/write_display_mode	DisplayModeState | OpError

The `GET /display/screenshot` route exists but refuses like its tool: framebuffer capture is unsolved below the message layer, so `screenshot` returns `rejected`, which the table maps to HTTP 409 (DES-028).

Pub-sub is not on REST. Publishing is scoped to the publisher's own session, and a REST caller shares one anonymous default scope with no subscribe route, so a REST publish could return success while being structurally unable to reach any subscriber. The pub-sub operations stay MCP-only until a REST caller has a real session identity to subscribe under. For the same reason, every REST call today lands in one default Hub scope; per-caller REST scoping arrives with the multi-user future.

9.3 The Command-Line Tool

The CLI is the third thin client of the one engine. `lux show beads` builds the beads element tree with the pure `BeadsBrowser` and sends it to luxd's REST API through `LuxRestClient` (`src/punt_lux/rest\_client.py`); it never touches the display socket. The client locates luxd's port from `HubPaths`, posts the request model, and reads the typed result. Two outcomes are distinct: luxd being unreachable (no port file, a refused connection, a stalled response) raises `HubUnavailableError` with an actionable message, while the Hub's refusal of a reachable request comes back as a typed `OpError` mapped from the HTTP status.

10 Performance

10.1 Frame Budget

Parameter	Value	Notes
Idle FPS	30	`fps_idling.fps_idle = 30.0`
Active FPS	60+	GLFW vsync, GPU-bound
Socket poll timeout	0	Non-blocking `select()`
Max message size	16 MiB	Hard limit in `FrameReader`
Connection timeout	5.0 s	`ensure()` and handshake
Session idle reap	1800 s	MCP session idle timeout
Frame-expiry poll	1.0 s	`ExpirySweep` idle cap
Texture loading	Lazy	First render triggers PIL + OpenGL upload
Event ring buffer	200 entries	Display recent interactions
Error ring buffer	100 entries	Display recent errors

10.2 Critical Path

A write does not wait on the Display. An operation returns as soon as it has updated `HubDisplay` and marked the scene dirty; the replicator does the socket send on its own thread, so a stuck or dead Display never blocks an agent-facing call. This liveness property — no mutator is ever disabled by display I/O — is one of the invariants the replicator formal model checks (Section §13).

The Display's per-frame cost is a `select()` on the one connection, a buffer drain, the ImGui draw calls (linear in visible element count), and an event flush. Table filtering is $O(R \cdot C)$ per frame in rows and columns, fast for tables under ∼10,000 rows, with pagination bounding the rendered rows.

10.3 Memory

- **Scene storage.** Scenes are Python dataclass trees in memory, bounded per message by the 16 MiB cap; dismissed scenes are garbage-collected.
- **Texture cache.** OpenGL texture IDs are held under an LRU policy capped at 256 entries (`LUX_TEXTURE_CACHE_CAP`); the least-recently-used texture is deleted on the GPU when a new upload would exceed the cap. The base64-payload→content-hash memo that keys data-sourced images is coupled to the same eviction, so it is bounded by the same cap rather than growing for the process lifetime. Both bound memory for image-heavy sessions.
- **Ring buffers.** The event and error buffers are bounded deques; constant memory.

10.4 Known Limitations

1. **No ImGui ID scoping.** Widget IDs derive from `element_id` alone, so ImGui internal state can bleed between tabs with colliding IDs.
2. **No screenshot capture.** Framebuffer capture is unsolved below the message layer, so `screenshot` refuses (DES-028).

11 Security

11.1 Trust Boundaries

There are two boundaries in the code as shipped today. The first is luxd's HTTP surface: it binds loopback, refuses a non-loopback bind at startup, and rejects a foreign `Host` or `Origin` header (Section §8). The second is the Display's Unix socket: luxd is its only client, filesystem permissions protect the socket directory (user-only, mode `0700`), and an oversize or malformed frame disconnects the peer rather than crashing the Display.

A third boundary is designed but not yet implemented: an opt-in, mutually authenticated TLS listener alongside the Unix socket, for the cross-host case where a Display aggregates Hubs on other machines. Section §7 (DES-090) specifies its threat model, its transport choice (TLS 1.3 against a per-user personal CA), and its invariants in full; it does not weaken, bypass, or share state with the two boundaries above.

11.2 Render Function Consent

A render function is the one element kind that executes arbitrary code, and its security model is consent-based, not sandbox-based. A best-effort AST scan flags suspicious imports and dangerous builtins as a UX signal, not a boundary; a consent modal shows the full source with any warnings, and the user must click Allow to run it; execution is not sandboxed — the code runs in the Display's process. A render function should be treated as user-approved code, equivalent to a script the user chose to run.

11.3 Protocol Robustness

- Malformed JSON disconnects the peer; it does not crash the Display.
- An unknown message type deserializes as `UnknownMessage` for forward compatibility.
- An oversize frame is rejected by `FrameReader` and converted to a disconnect.

- A patch that tries to change an element's `id` or `kind` is dropped.
- Double-remove of a client is idempotent.

12 Platform Integration

12.1 macOS

- The Display runs as a normal Dock app; the Dock icon and Cmd-Tab entry aid operational debugging.
- `setproctitle("Lux")` names the process in `ps` and `top`.
- Font discovery prefers Arial Unicode or Helvetica and merges Apple Symbols and STIX Two Math for mathematical glyphs.
- Socket paths stay within the ∼104-character `AF_UNIX` limit; tests use `tempfile.mkdtemp(prefix="lux-")`.

12.2 Linux

- Font discovery uses DejaVu Sans or Noto Sans and merges Noto Sans Symbols2 and Noto Sans Math.
- Socket placement prefers `$XDG_RUNTIME_DIR/lux/`.
- Window managers use the X11 window title ("Lux").

13 Formal Models

Four Z specifications hold parts of the system to a checked model. Each is type-checked with Spivey's `fuzz` and model-checked with ProB; the first two model concurrency the current code runs, and the third models the Display singleton lifecycle. The fourth is the legacy display-server model, which the current display code is still refined against.

Model	What it proves
`docs/hub_replicator.tex`	The single-writer, two-lock replication discipline. Six invariants I1–I6: only the replicator writes to the Display (single writer); no mutator is ever blocked by display I/O (agent-path liveness); no thread holds the store lock and the send lock at once (no lock cycle); the display shows the store's latest state at rest, including after a reap and respawn (no lost update); a respawn follows a kill (one display); and the menu bar is repainted before rest (menu durability).
`docs/frame_expiry.tex`	The re-show / expiry atomicity invariant: a frame re-armed with a fresh TTL is never torn down by a stale deadline, because both the re-show and the sweep run under the one store lock. The fidelity variant drops the atomicity and ProB reaches the exact bad state where a freshly re-shown frame has been torn down (Section §5).
`docs/display_lifecycle.tex`	The Display-singleton spawn lifecycle (DES-037/038): at most one Display exists across concurrent spawn attempts, and the lifecycle is deadlock-free.
`docs/architecture/display-server.tex`	The legacy display-server state machine: client connect and disconnect, scene replacement, patching, and event flush. Amended so that an empty-element push is a scene removal, matching the frames-die-with-scenes behavior (Section §5); the refinement harness now drives an empty push and holds the code to the amended model.

Refinement tests in `tests/test_display_refinement.py` tie the concrete Python back to the legacy model through an abstraction function, and partition tests in `tests/test_display_partition.py` derive cases from its operation schemas. The legacy model predates multi-scene tabs and the Hub/Display split; its abstraction maps multi-scene state to the active tab.

Pending, not yet built. DES-090's cross-host design (§7) names two invariants as z-spec-REQUIRED once implementation starts — no content before verification, and at most one live connection per `HubId` — with the concrete race each model must exhibit and exclude, and a fidelity requirement for each. Neither is modeled yet; the design mission's own instruction was to flag, not model, ahead of implementation. Tracked in the increment-of-work document, `docs/architecture/multi-hub-addressing-work.md`.

14 Test Architecture

The suite is layered; the exact count changes, so this records the shape.

- **Protocol and codec.** Framing, serialization roundtrips, typed draw-command decode, value validation, and self-validation (DES-039). Every element kind has a Level-1 roundtrip; a protocol change without one is unshippable.
- **Operations and surfaces.** The operations facade over fakes, the REST routers via `TestClient`, the MCP adapters, and the CLI's `LuxRestClient`.
- **Hub domain.** `HubDisplay` authority and ownership, the D21 dispatch, the frame lifecycle, and the replicator.
- **Guards.** The display-socket AST guard (§8) and the MCP-tool characterization corpus, which replays a captured snapshot per tool so the string contract cannot drift silently.
- **Formal-model alignment.** The refinement and partition tests against the legacy display-server model.
- **Integration and e2e.** Marker-gated tests exercise the real Hub/Display boundary; the business-event-loop harness (`tests/e2e/`) proves the full bidirectional D21 loop for a composed surface across the faithful in-memory connection.

Quality gates run through the Makefile: `ruff` (lint and format), `mypy` and `pyright` (both strict), `pytest`, `check-oo` (the OO ratchet against `.oo-baseline.json`), and the snapshot-parity gate. `make check` is the composite gate.